



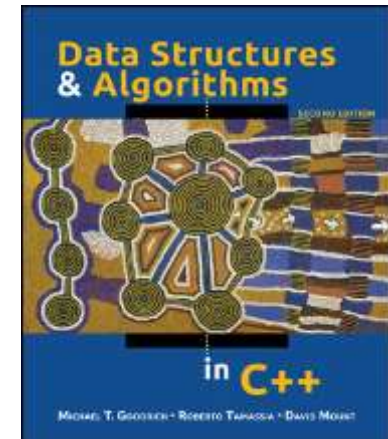
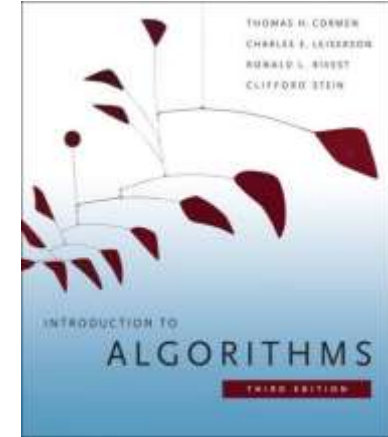
CSE 2215: Data Structure and Algorithms-I

Introduction, Complexity Analysis

The Course

Text Books

- Introduction to Algorithms (Third edition)
 - Cormen, Leiserson, Rivest, and Stein
 - An excellent reference **you should own**
- Data Structures and Algorithms in C++
 - Goodrich, Tamassia, and Mount



Contents

- Introduction to Data Structures
- Introduction to Algorithms
- Analysis of Algorithms



INTRODUCTION TO DATA STRUCTURES

What is a Data Structure?

- Data is a **collection** of facts, such as values, numbers, words, measurements, or observations.
- Structure means a **set of rules** that holds the data together.
- A data structure is a particular way of storing and organizing data in a computer so that it can be used efficiently.
 - **Different kinds** of data structures are suited to different kinds of applications, and some are highly specialized to specific tasks.
 - Data structures provide a means to manage **huge amount** of data efficiently.

Types of Data Structures

- Data structures are classified as either
 - **Linear** (e.g, arrays, linked lists), or
 - **Nonlinear** (e.g, trees, graphs, etc.)
- A data structure is said to be linear if it satisfies the following four conditions
 - There is a unique element called the **first**
 - There is a unique element called the **last**
 - Every element, except the last, has a unique **successor**
 - Every element, except the first, has a unique **predecessor**
- There are two ways of representing a linear data structure in memory
 - By means of sequential memory locations (**arrays**)
 - By means of pointers or links (**linked lists**)



INTRODUCTION TO ALGORITHMS

What is an Algorithm?

- An algorithm is a **sequence of computational steps** that transform the input into the output.
- An algorithm is a tool for solving a well-specified computational problem.
 - An algorithm is said to be **correct** if, for every input instance, it **halts** with the correct output
 - An incorrect algorithm might not halt at all on some input instances, or it might halt with other than the desired output.
- Note that algorithms existed **way before the invention of computers**

What is a Program?

- A program is the expression of an algorithm in a programming language
- A set of instructions which the computer will follow to solve a problem



Algorithm vs Program

INSERTION-SORT(A)

```
1  for  $j = 2$  to  $A.length$ 
2       $key = A[j]$ 
3      // Insert  $A[j]$  into the sorted sequence  $A[1..j-1]$ .
4       $i = j - 1$ 
5      while  $i > 0$  and  $A[i] > key$ 
6           $A[i+1] = A[i]$ 
7           $i = i - 1$ 
8       $A[i+1] = key$ 
```

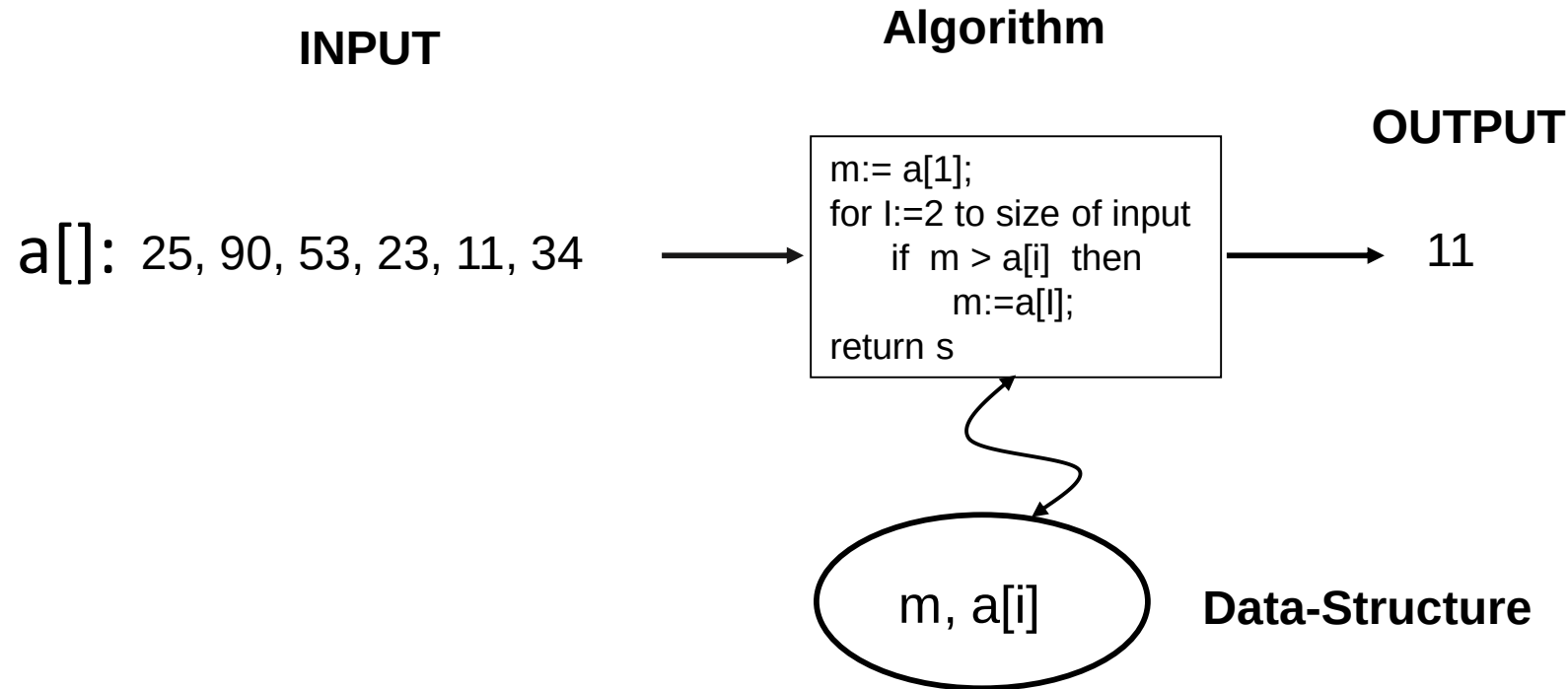
```
#include<iostream>
using namespace std;
int main () {
    int A[10] = {112,14,34,91,15,48,53,1,0,23};
    cout<<"List Before Sort: ";
    for(int i=0;i<10;i++) {
        cout <<A[i]<<" ";
    }
    for(int k=1; k<10; k++) {
        int temp = A[k];
        int j= k-1;
        while(j>=0 && temp <= A[j]) {
            A[j+1] = A[j];
            j = j-1;
        }
        A[j+1] = temp;
    }
    cout<<"\nList After Sort: ";
    for(int i=0;i<10;i++) {
        cout <<A[i]<<" ";
    }
    cout<<"\n";
}
```

The Big Picture

Problem:

Input is a sequence of integers stored in an array.

Output the minimum.





ANALYSIS OF ALGORITHM

Necessity of Analysis of Algorithms

- Need to recognize **limitations** of various algorithms for solving a problem
- Need to understand relationship between problem size and **running time**
- Need to learn how to analyze an algorithm's running time **without coding it**
- Need to recognize **bottlenecks** in code as well as which parts of code are easiest to optimize
- Need to learn techniques for writing more **efficient code**

What do we Analyze in an algorithm?

- Correctness
 - Does the input/output relation match algorithm requirement?
- Amount of work done (complexity)
 - Basic operations to do task
- Amount of space used
 - Memory used
- Simplicity, clarity
 - Verification and implementation.
- Optimality
 - Is it impossible to do better?

Analysis Based on Input Size

- **Time and space complexity**
 - This is generally a function of the **input size**
 - e.g., sorting, multiplication
 - How we characterize input size depends
 - Sorting: number of **input elements**
 - Graph algorithms: **number of nodes & edges**
 - Etc.

Insertion Sort

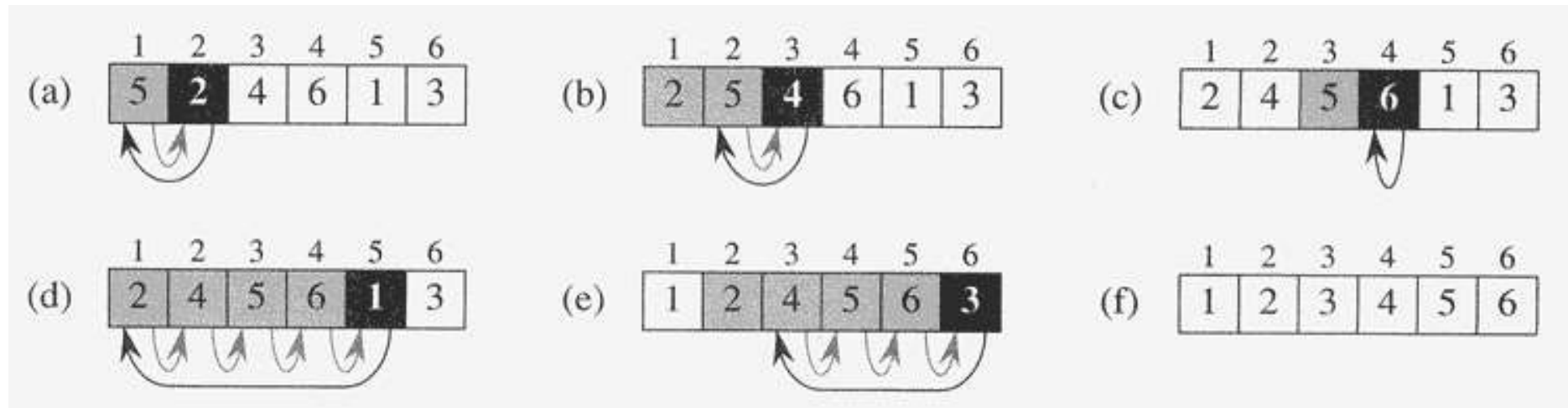
Input: A sequence of n numbers $\langle a_1, a_2, \dots, a_n \rangle$.

Output: A permutation (reordering) $\langle a'_1, a'_2, \dots, a'_n \rangle$ of the input sequence such that $a'_1 \leq a'_2 \leq \dots \leq a'_n$.

WHERE IS THE ALGORITHM?

An Example: Insertion Sort

$A = \{5, 2, 4, 6, 1, 3\}$



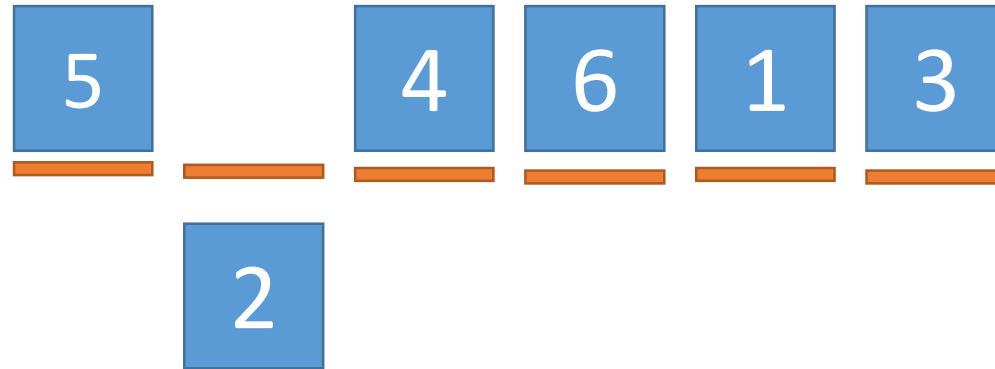
Analyzing Insertion Sort

$i=2$

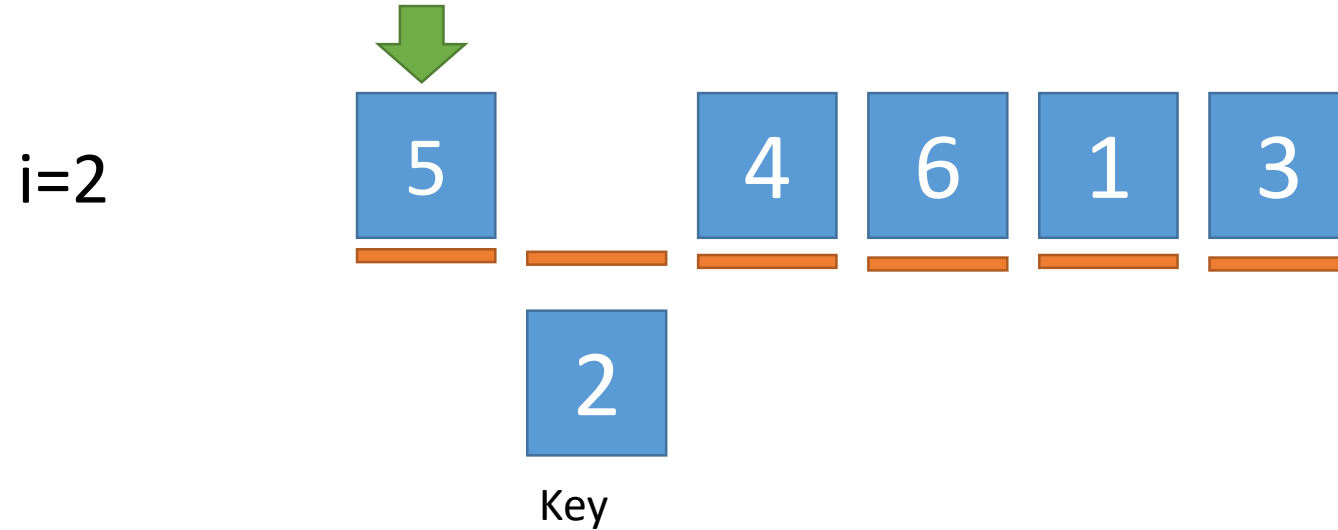


Analyzing Insertion Sort

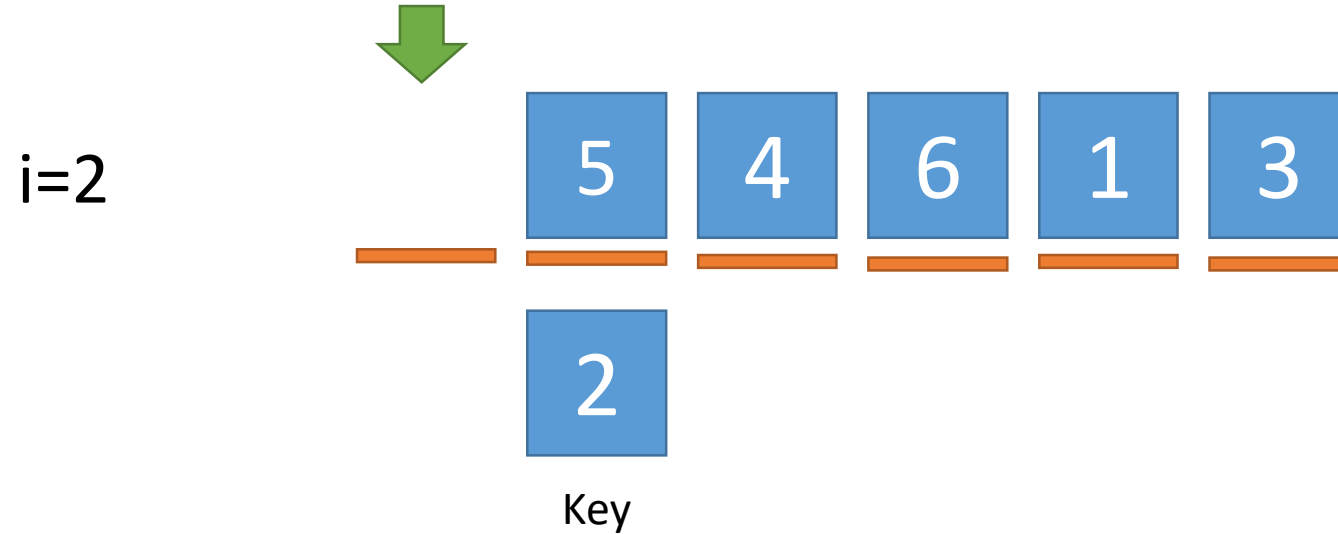
$i=2$



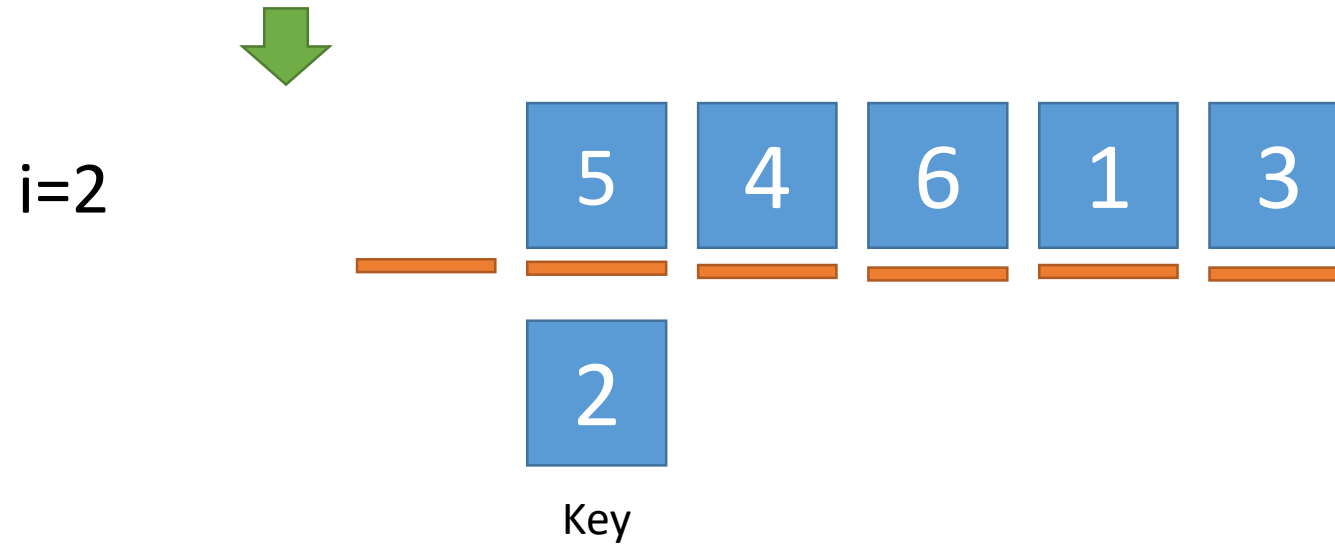
Analyzing Insertion Sort



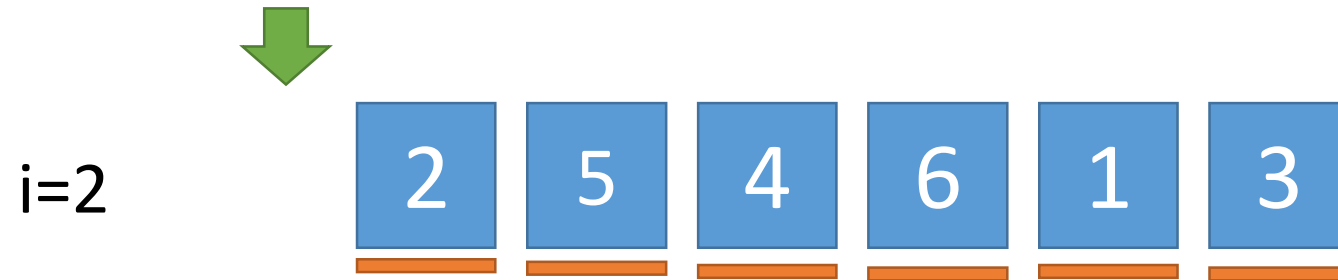
Analyzing Insertion Sort



Analyzing Insertion Sort



Analyzing Insertion Sort

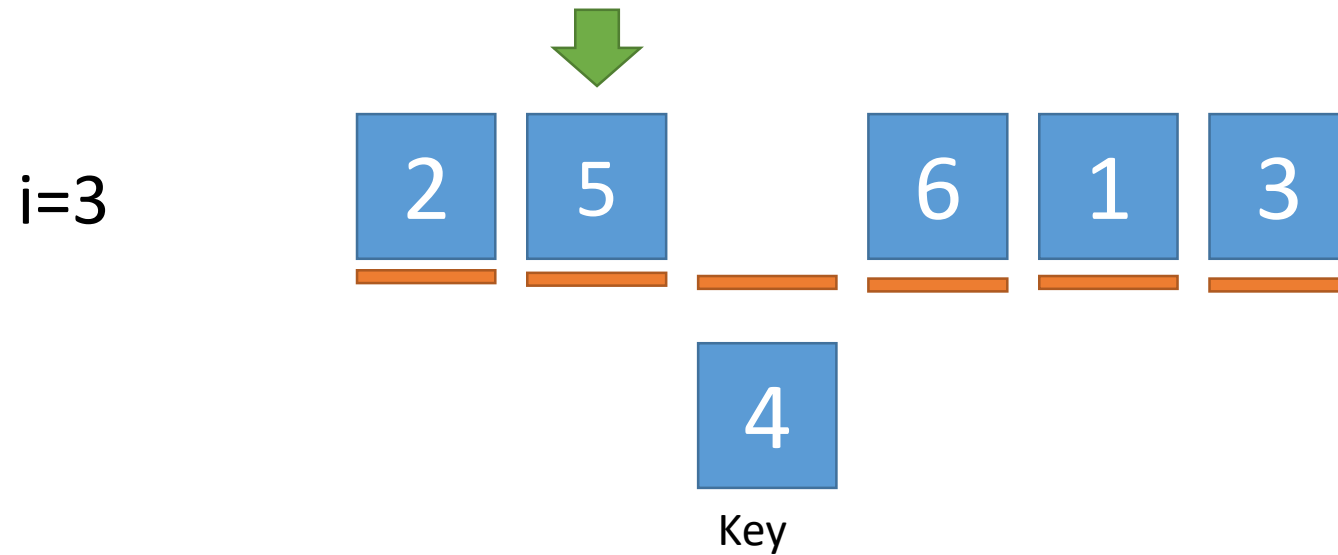


Analyzing Insertion Sort

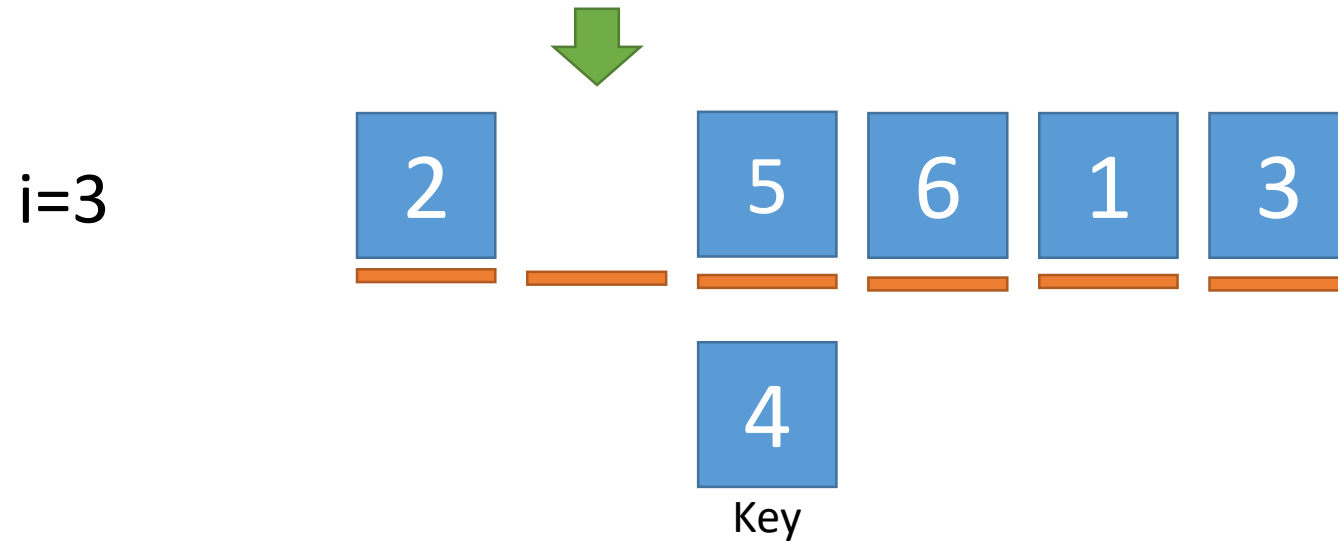
$i=3$



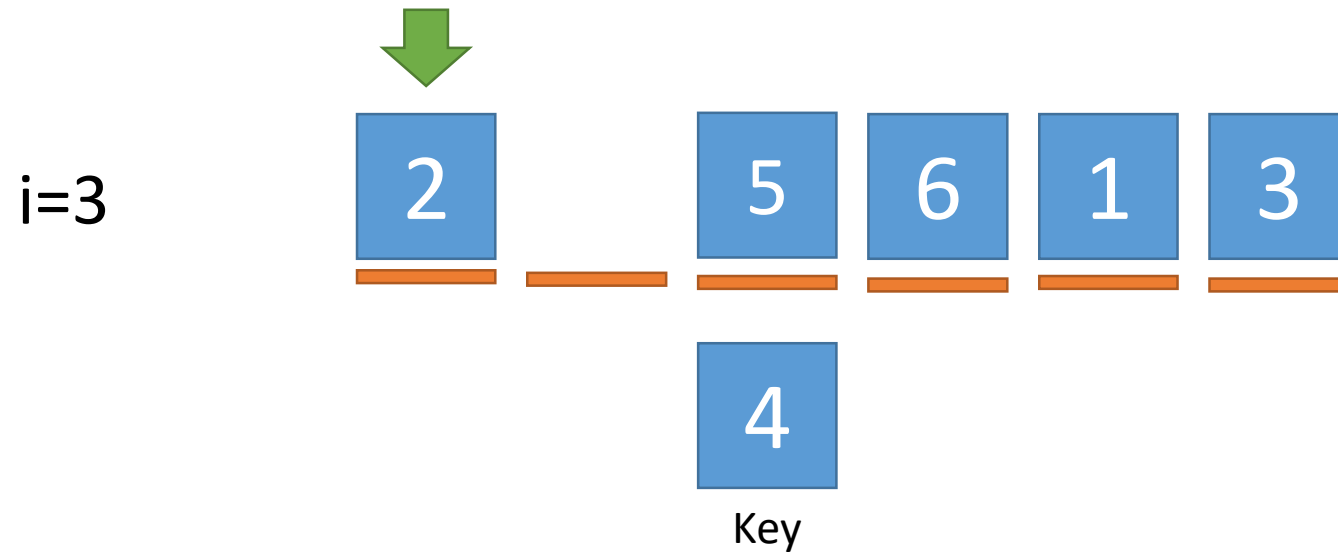
Analyzing Insertion Sort



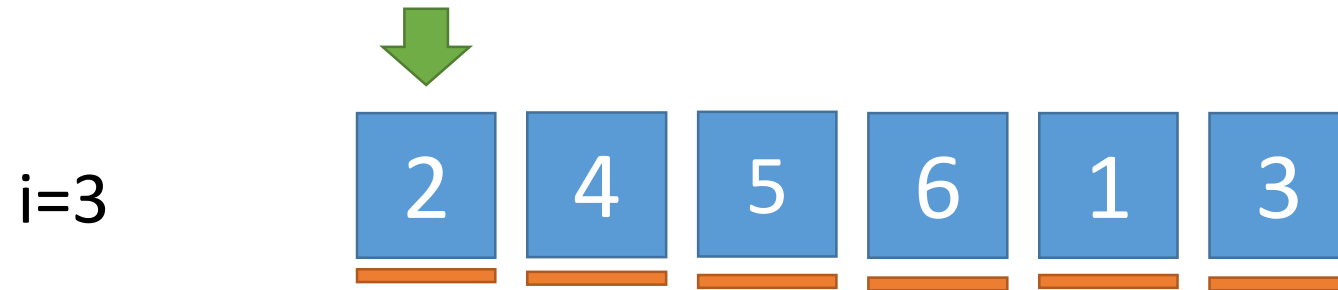
Analyzing Insertion Sort



Analyzing Insertion Sort



Analyzing Insertion Sort

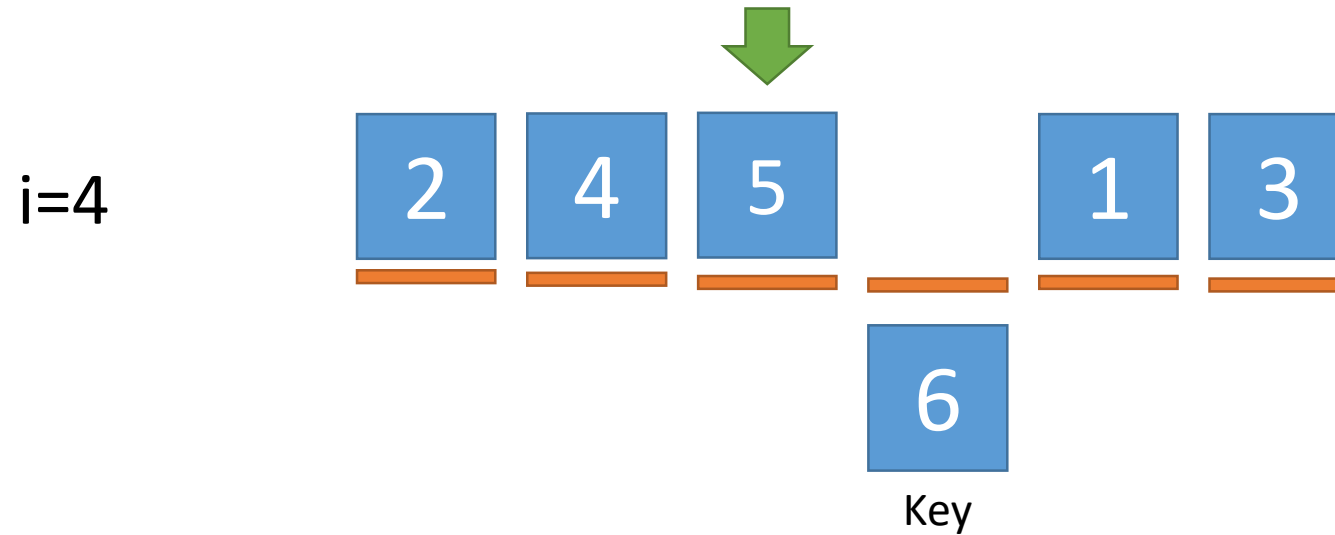


Analyzing Insertion Sort

$i=4$



Analyzing Insertion Sort

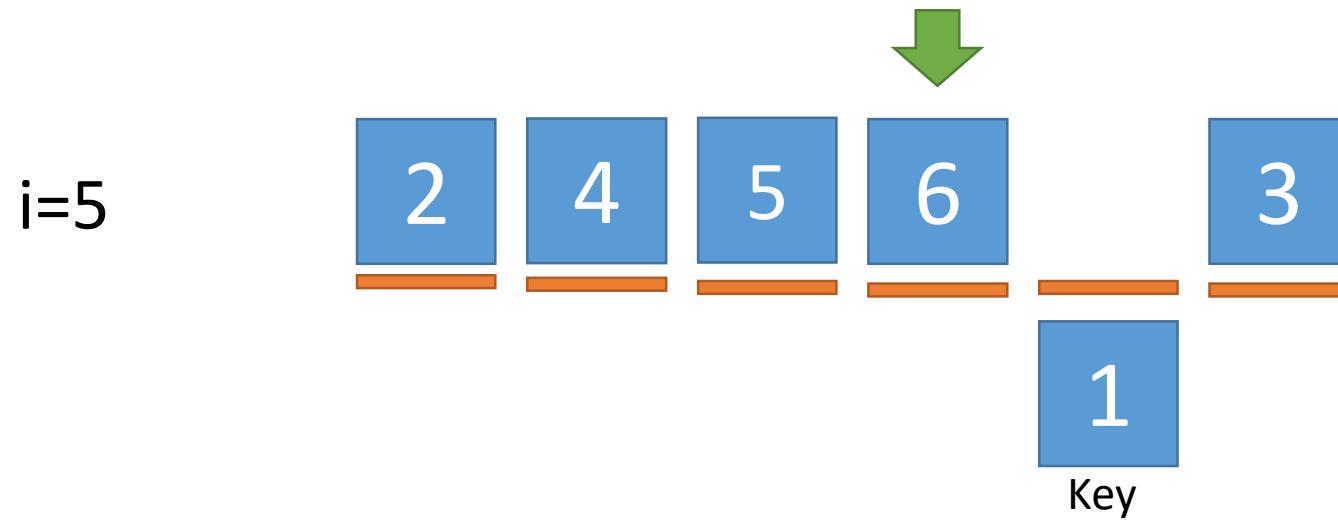


Analyzing Insertion Sort

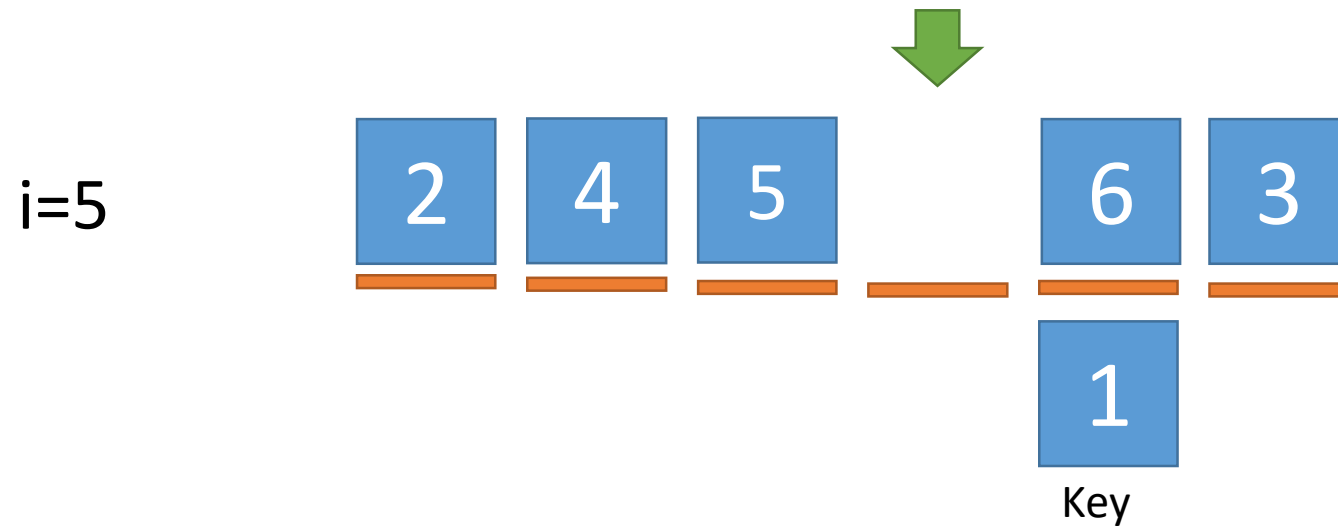
$i=4$



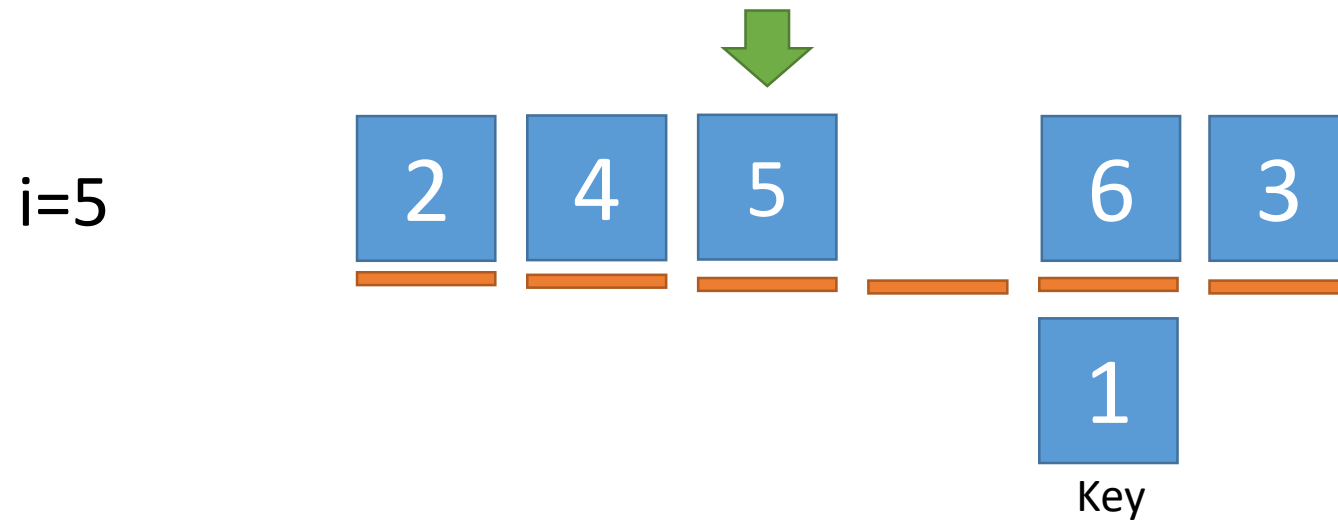
Analyzing Insertion Sort



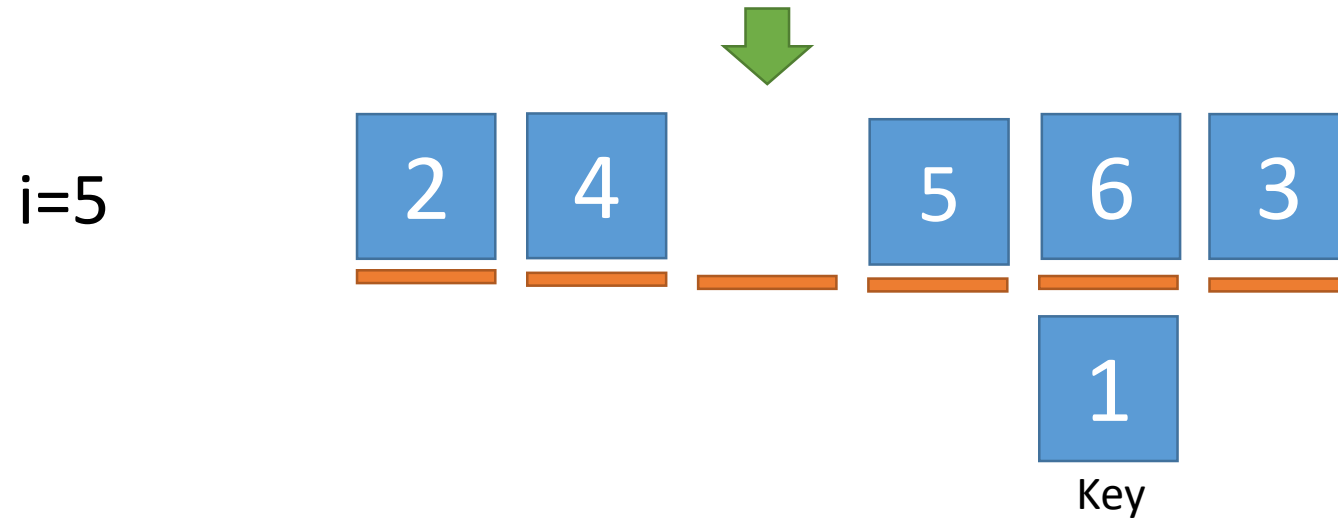
Analyzing Insertion Sort



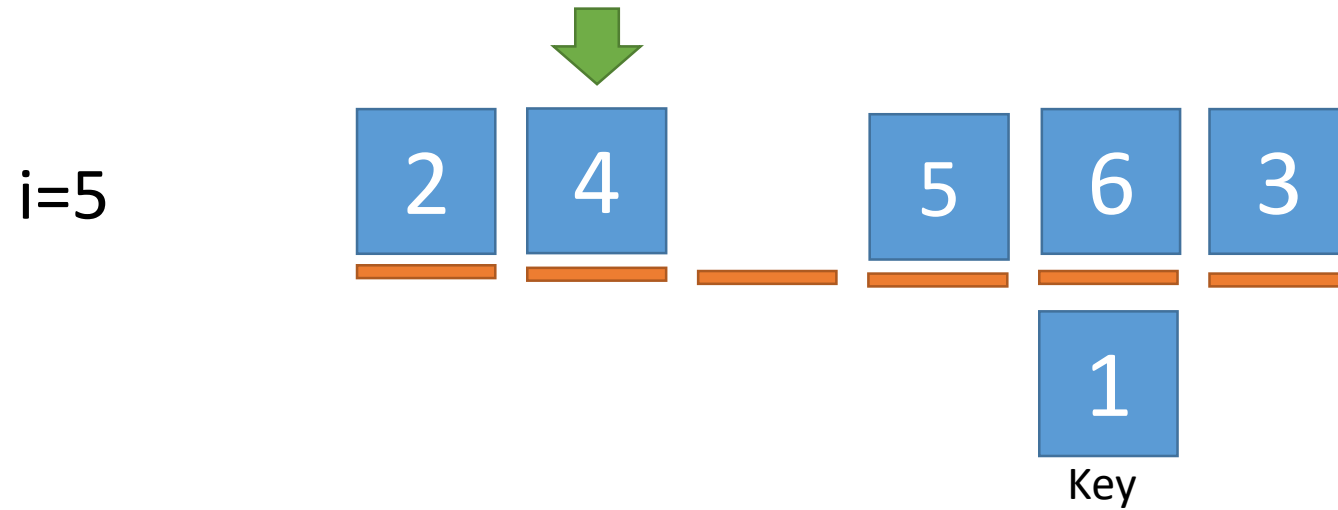
Analyzing Insertion Sort



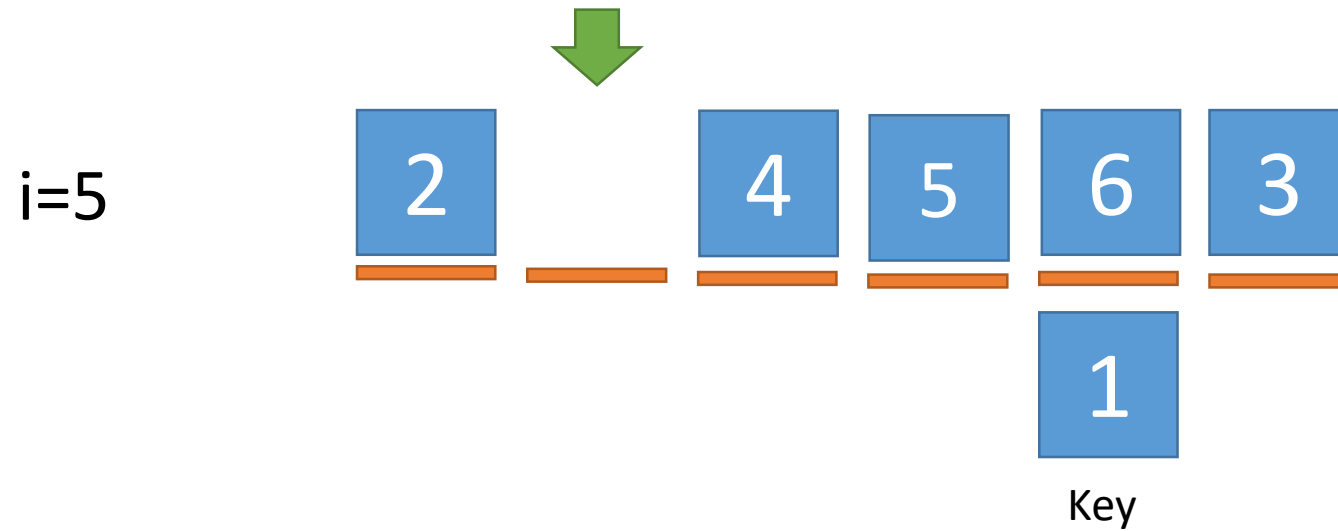
Analyzing Insertion Sort



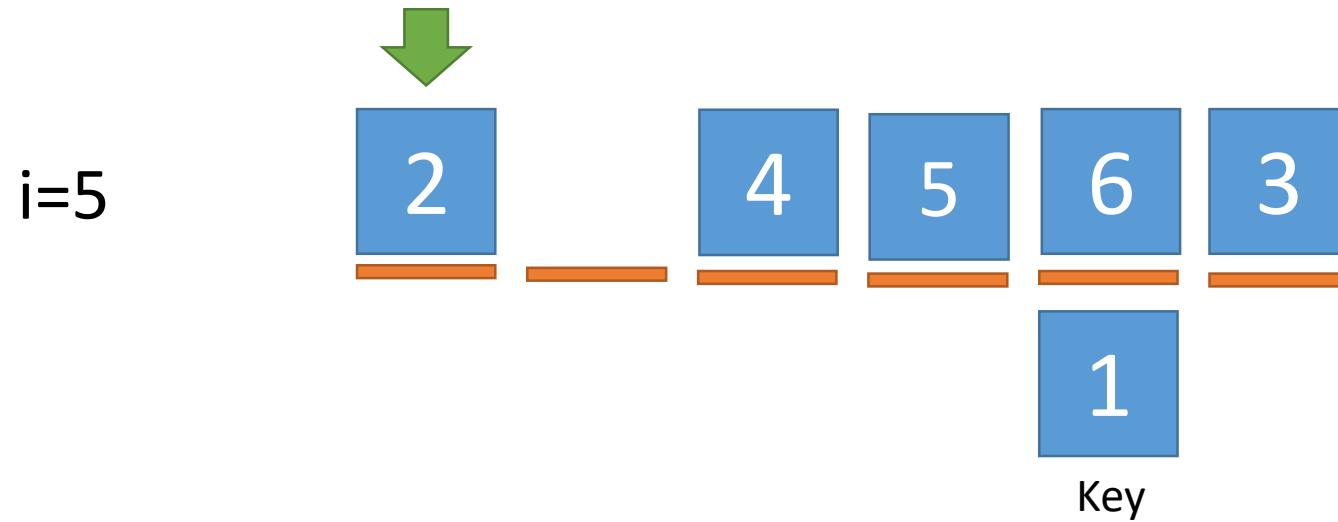
Analyzing Insertion Sort



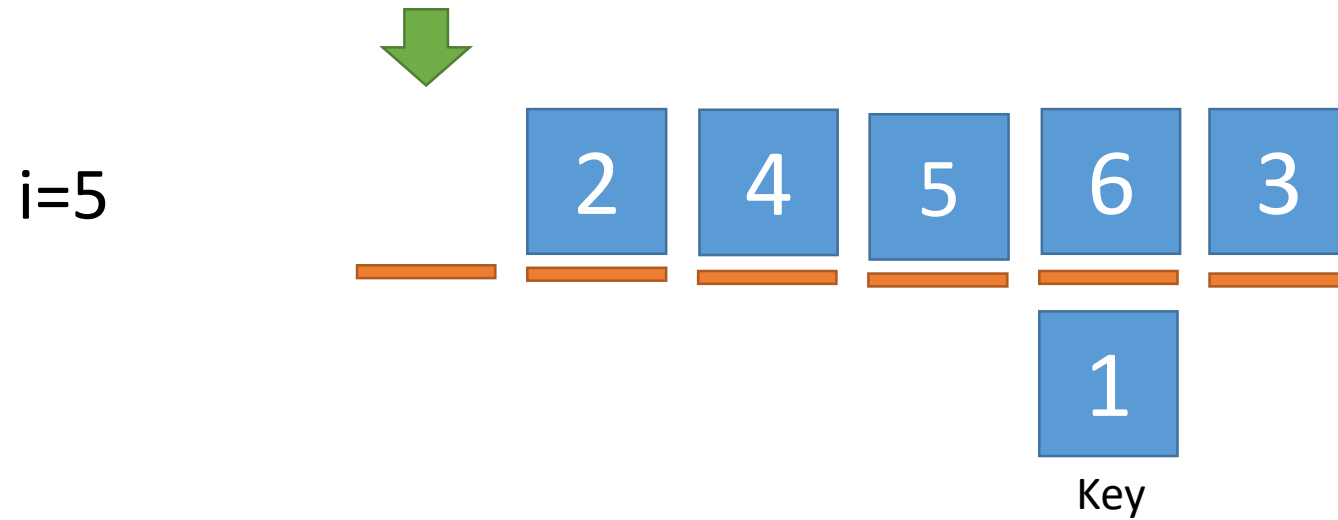
Analyzing Insertion Sort



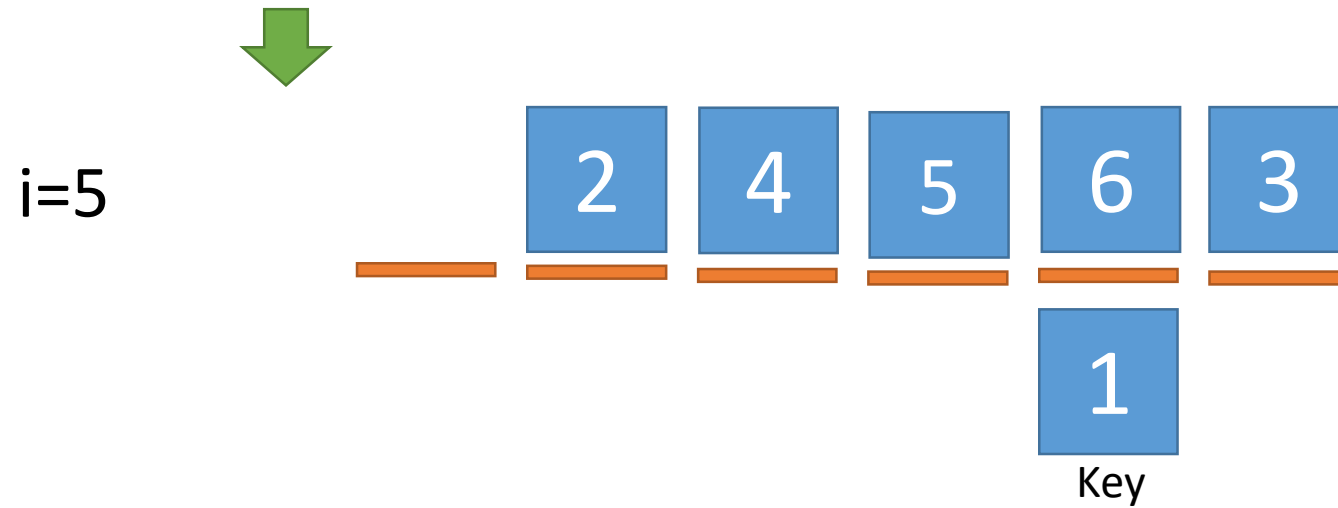
Analyzing Insertion Sort



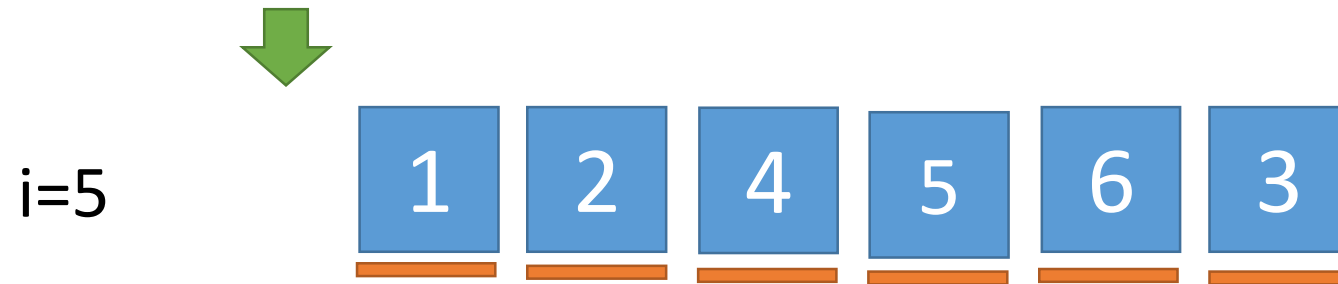
Analyzing Insertion Sort



Analyzing Insertion Sort



Analyzing Insertion Sort



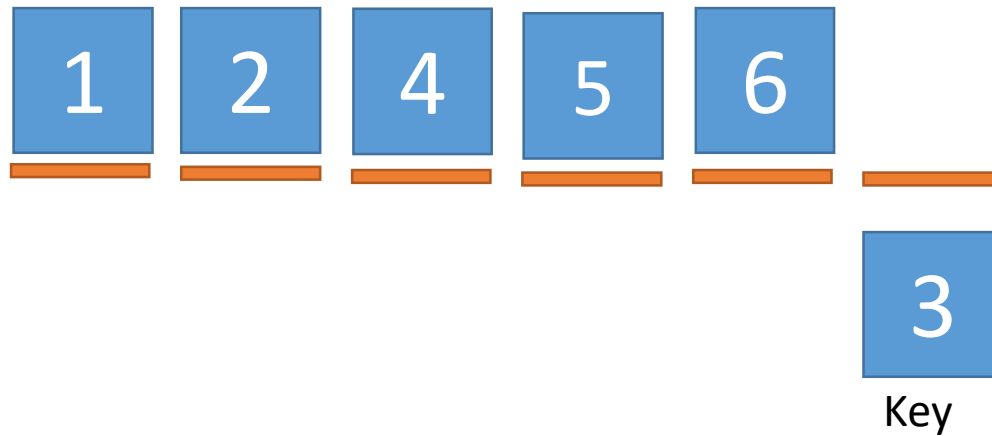
Analyzing Insertion Sort

$i=6$

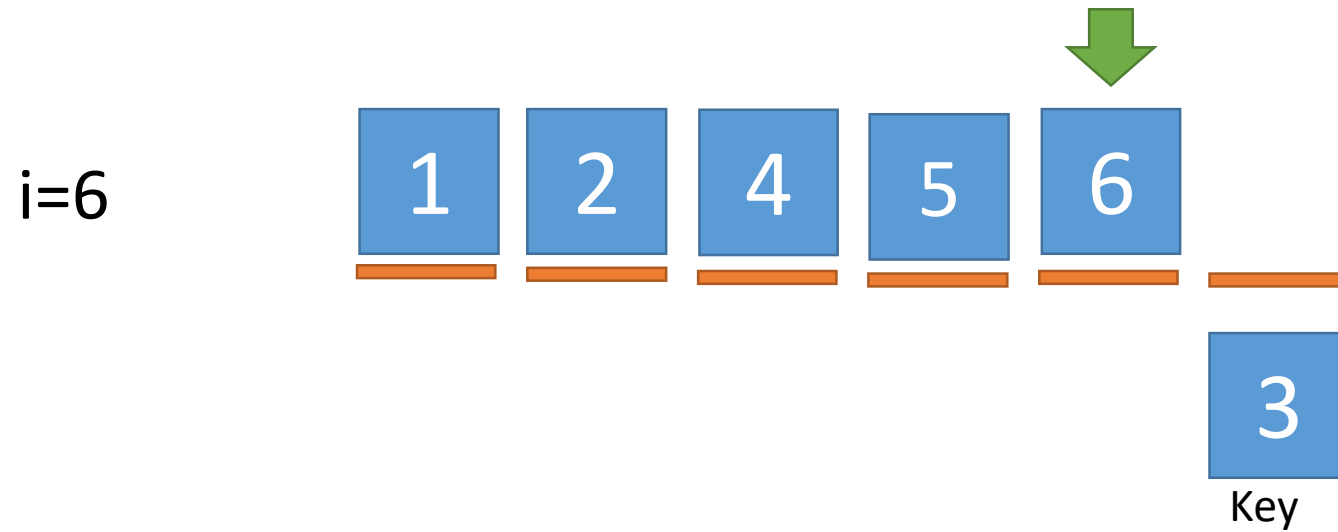


Analyzing Insertion Sort

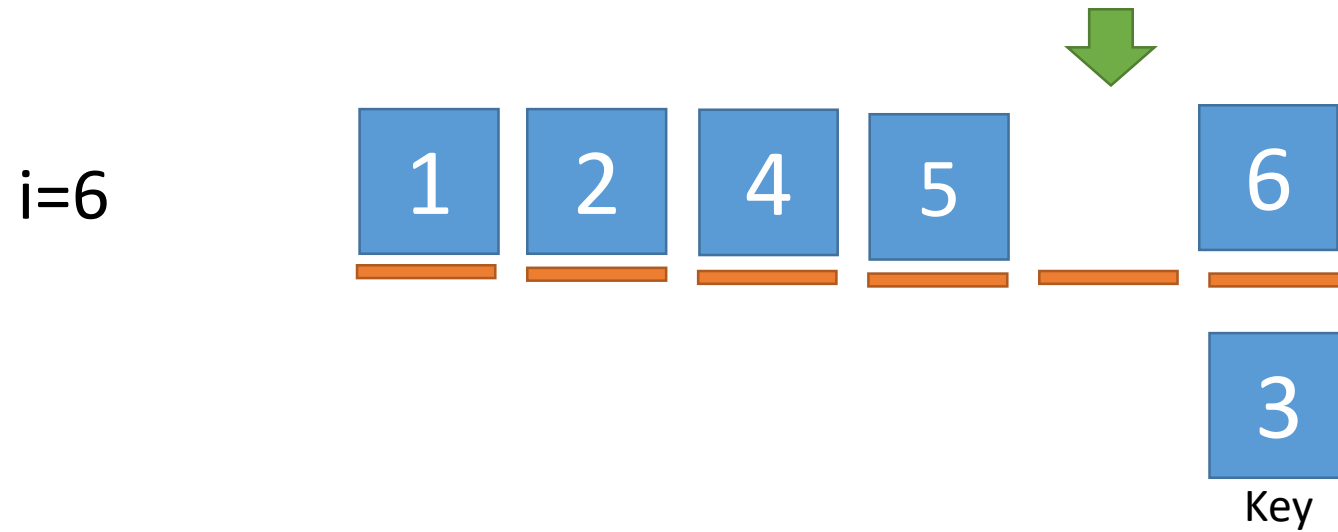
i=6



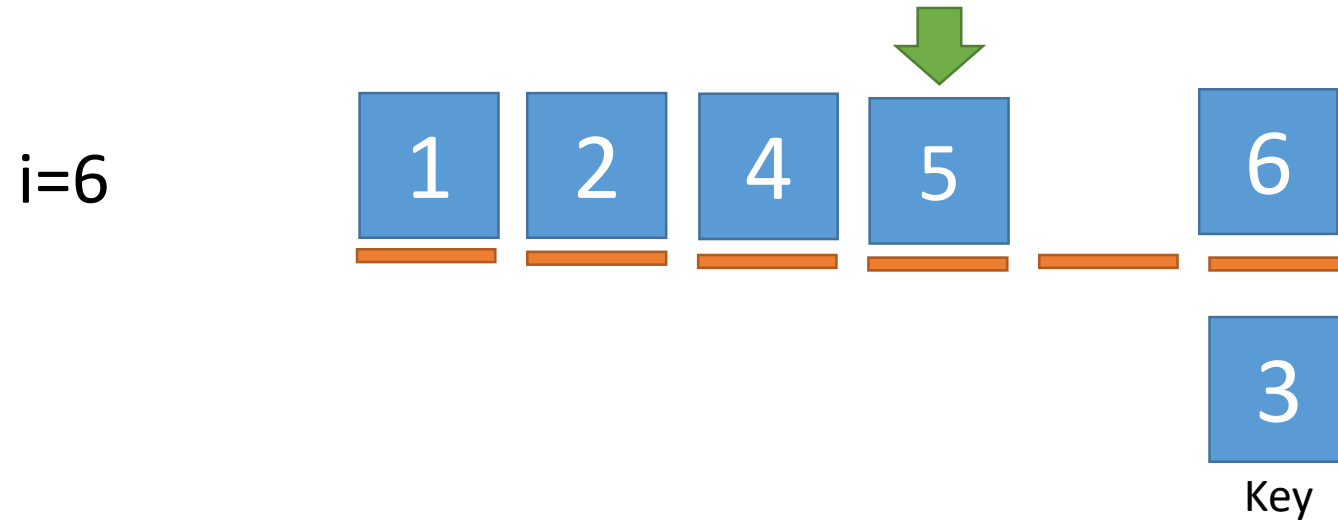
Analyzing Insertion Sort



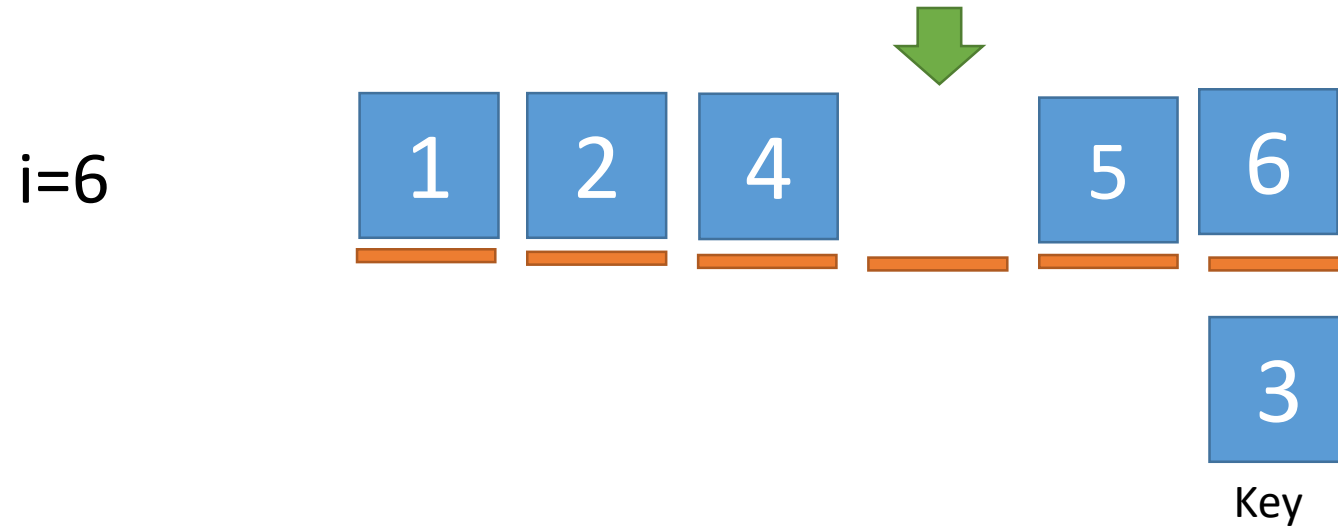
Analyzing Insertion Sort



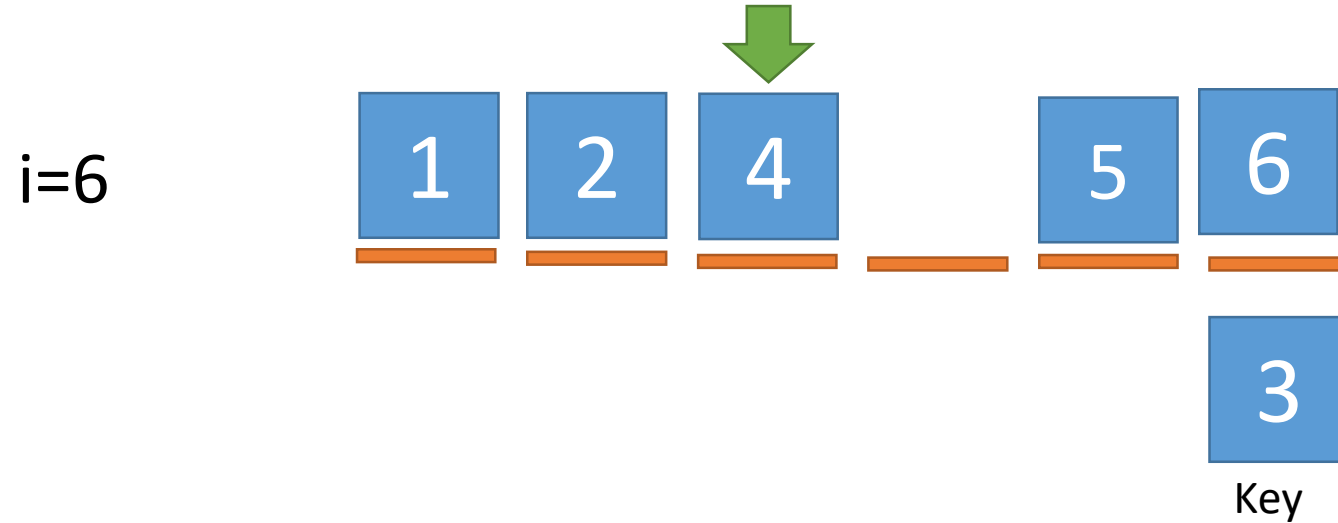
Analyzing Insertion Sort



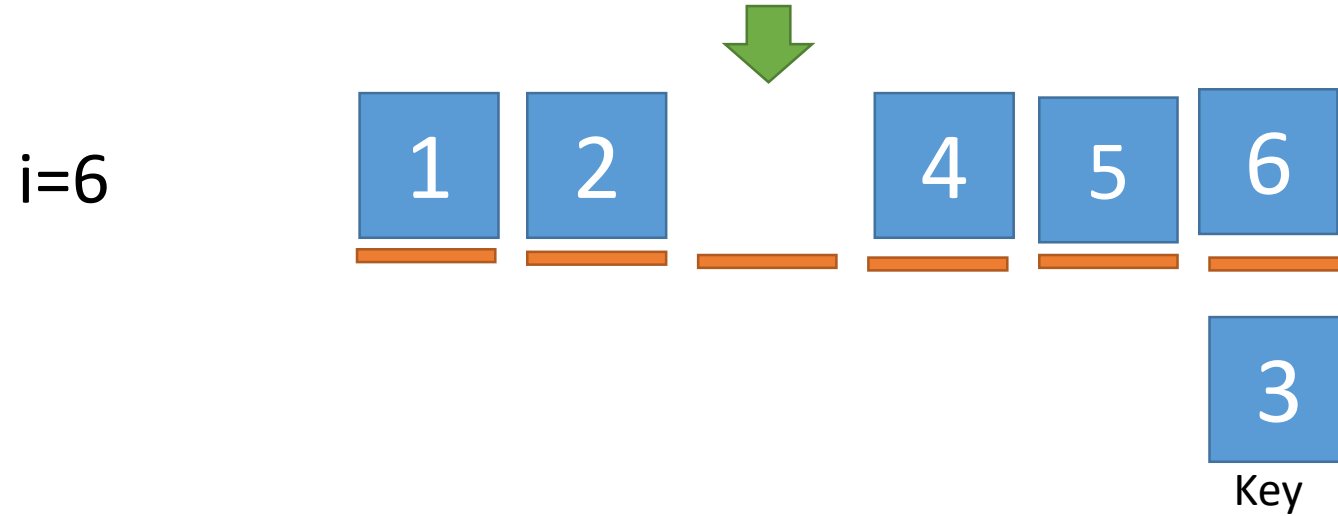
Analyzing Insertion Sort



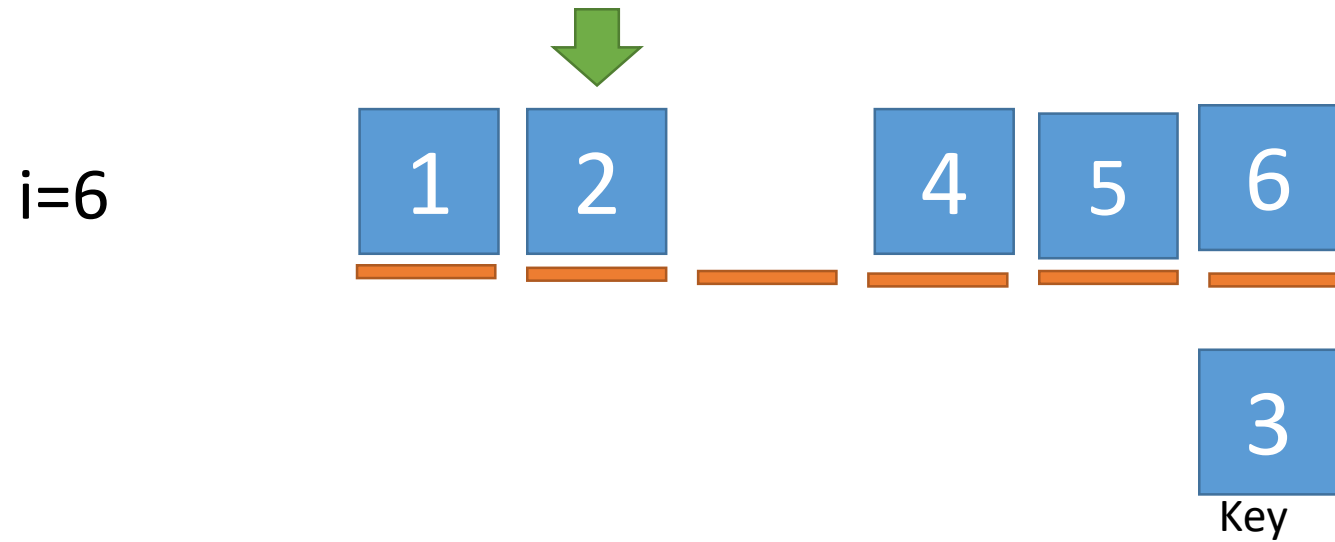
Analyzing Insertion Sort



Analyzing Insertion Sort



Analyzing Insertion Sort



Analyzing Insertion Sort

$i=6$



Analyzing Insertion Sort

Now let us try to write the Algorithm (in pseudocode format)

Analyzing Insertion Sort

Now let us try to write the Algorithm (in pseudocode format)

```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1 }  
        A[j+1] = key  
    }  
}
```

Analyzing Insertion Sort

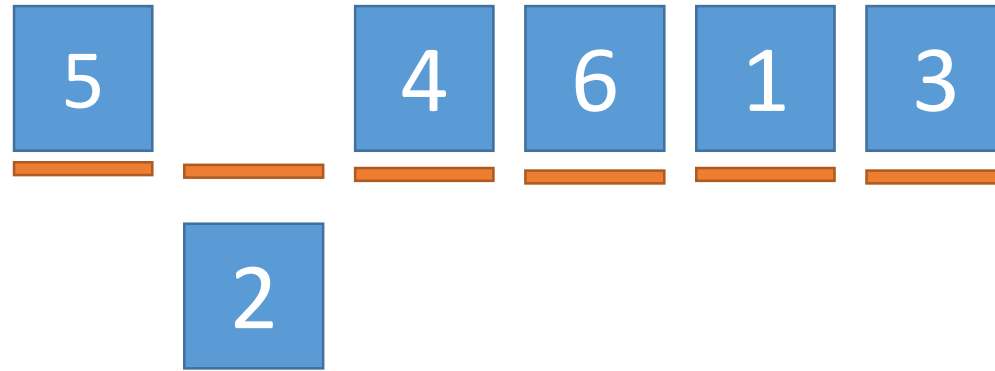
$i=2$



```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

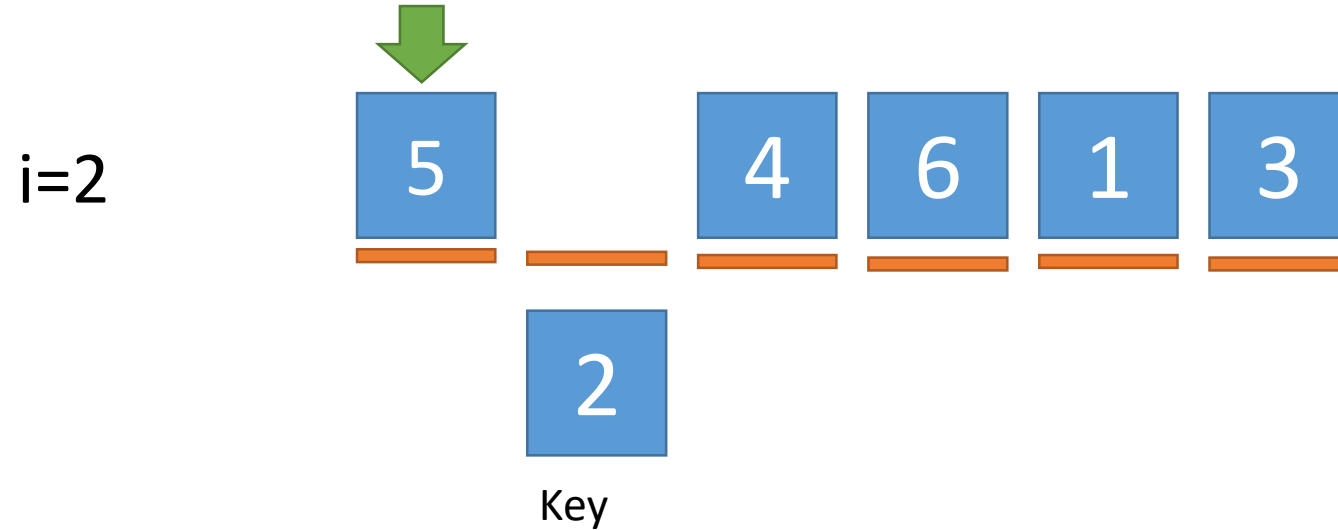
Analyzing Insertion Sort

$i=2$



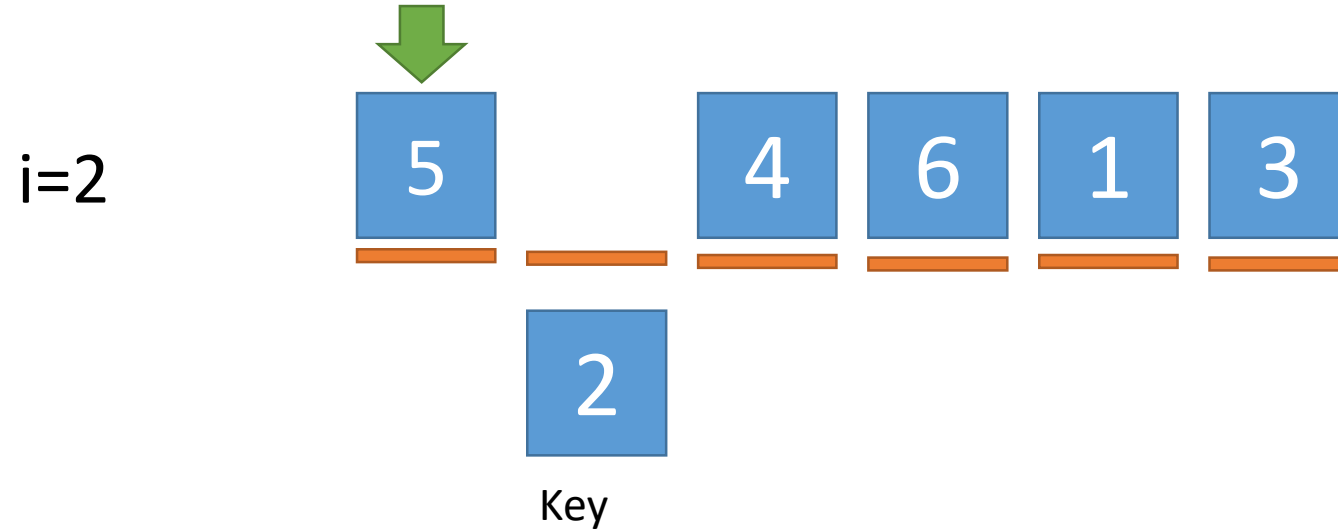
```
InsertionSort(A, n) {  
  for i = 2 to n {  
    → key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1 }  
    A[j+1] = key  
  }  
}
```

Analyzing Insertion Sort



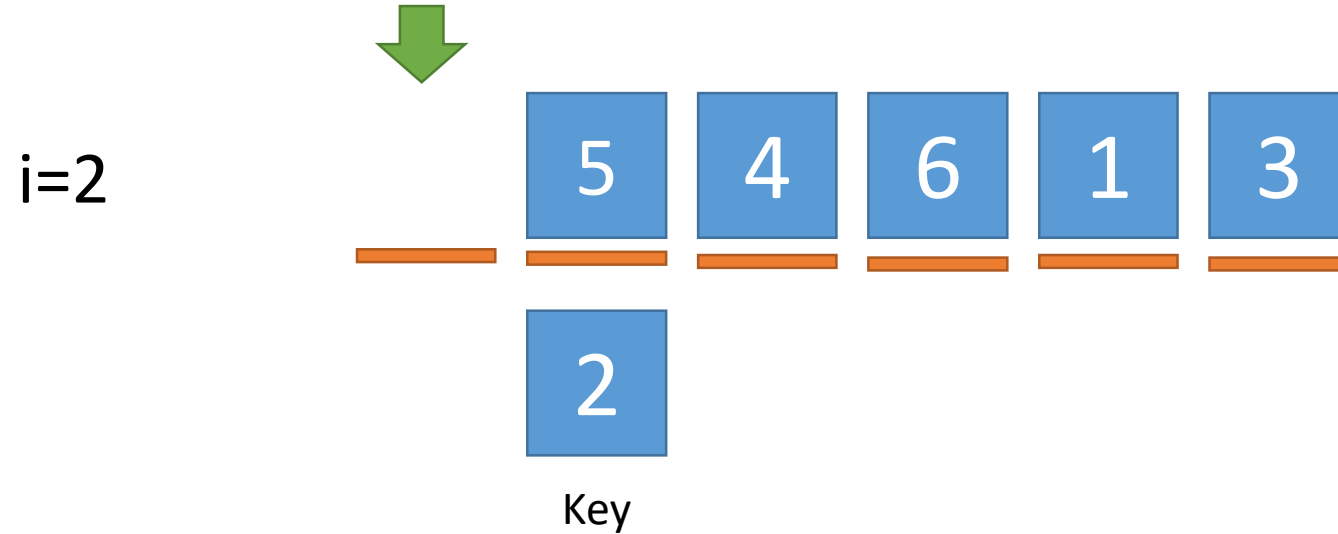
```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        → j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```


Analyzing Insertion Sort



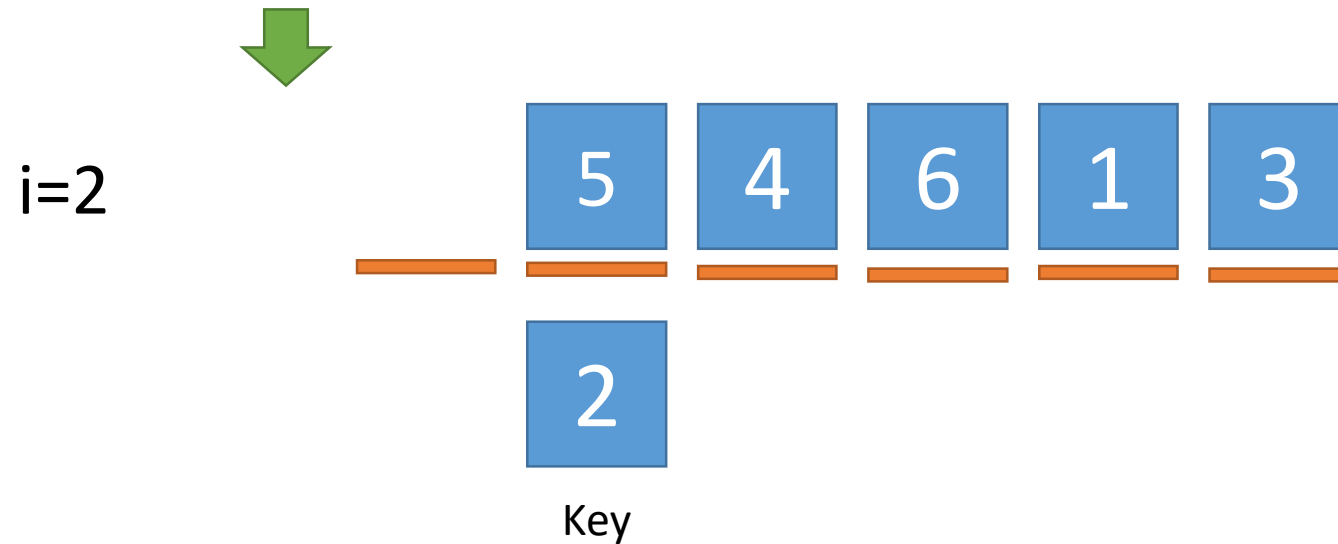
```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        → while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1 }  
        A[j+1] = key  
    }  
}
```

Analyzing Insertion Sort



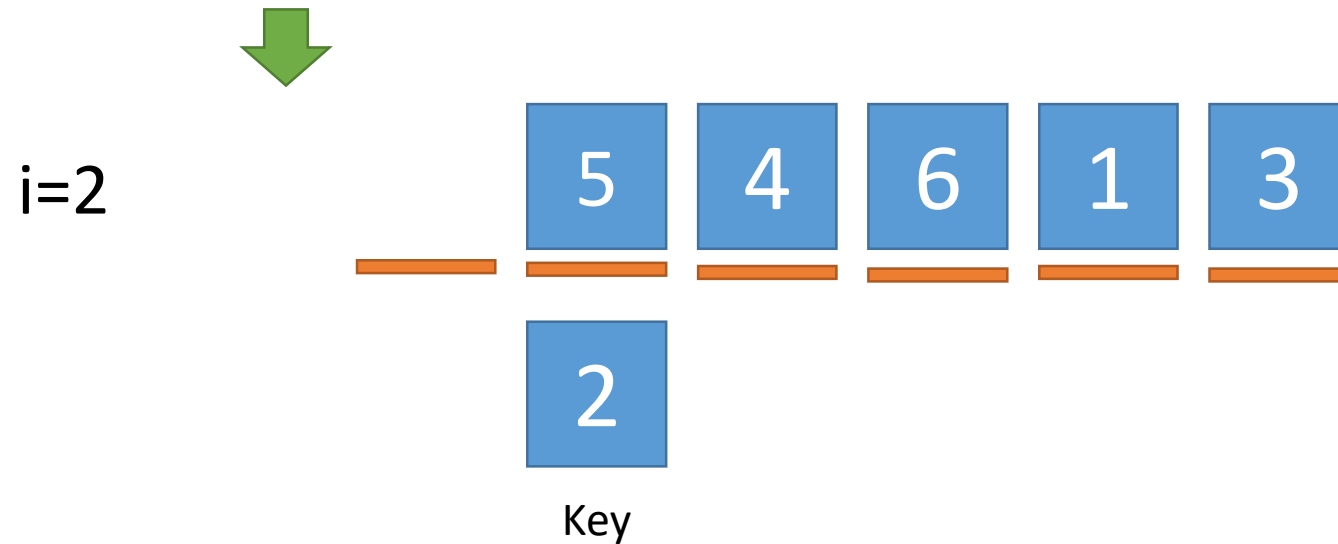
```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
       A[j+1] = A[j]  
      j = j - 1  
    }  
    A[j+1] = key  
  }  
}
```

Analyzing Insertion Sort



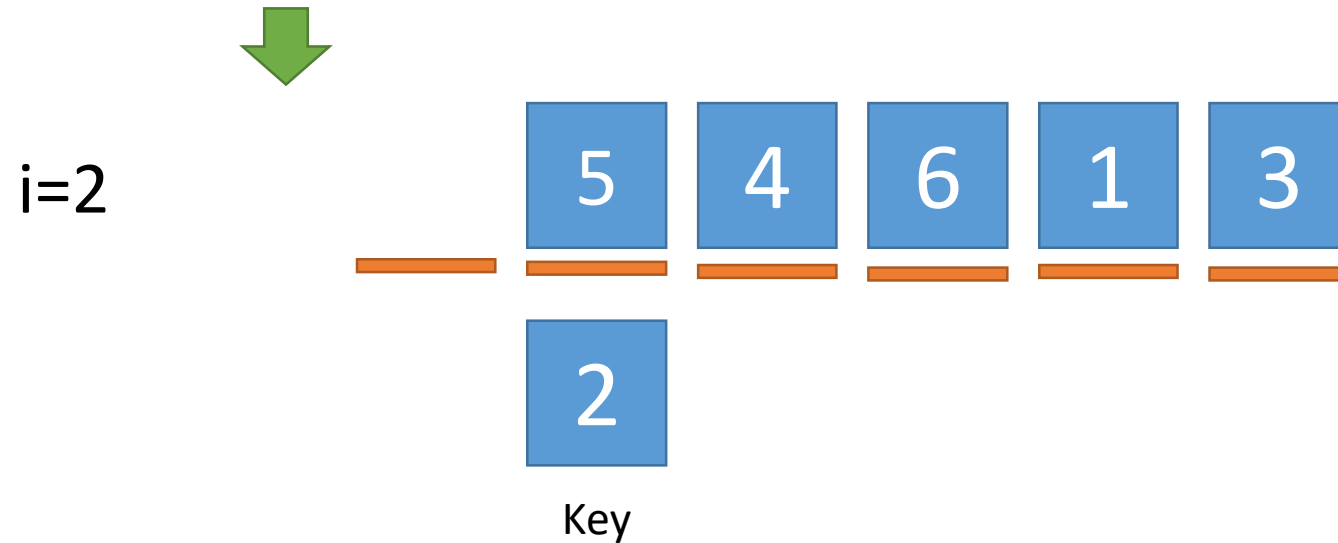
```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      → j = j - 1  
    }  
    A[j+1] = key  
  }  
}
```

Analyzing Insertion Sort



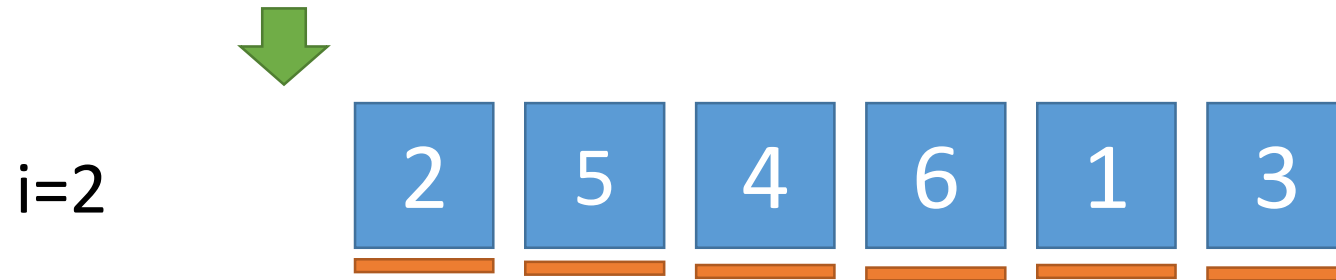
```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    → while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1 }  
    A[j+1] = key  
  }  
}
```

Analyzing Insertion Sort



```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    → while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1 }  
    A[j+1] = key  
  }  
}
```

Analyzing Insertion Sort



```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1 }  
    → A[j+1] = key  
  }  
}
```

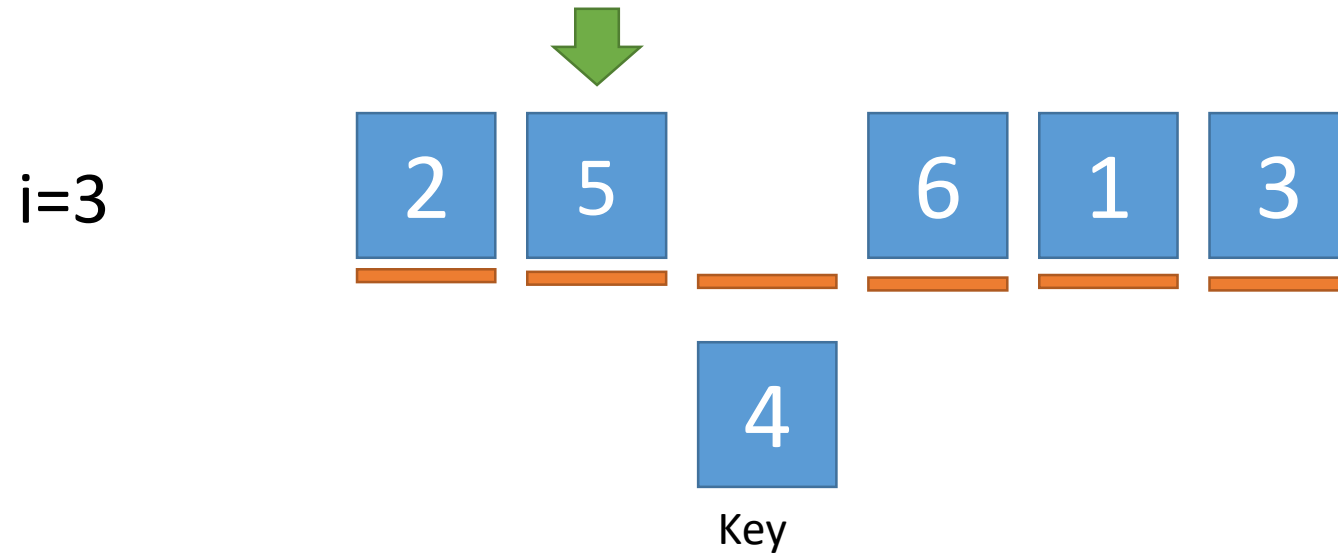
Analyzing Insertion Sort

$i=3$



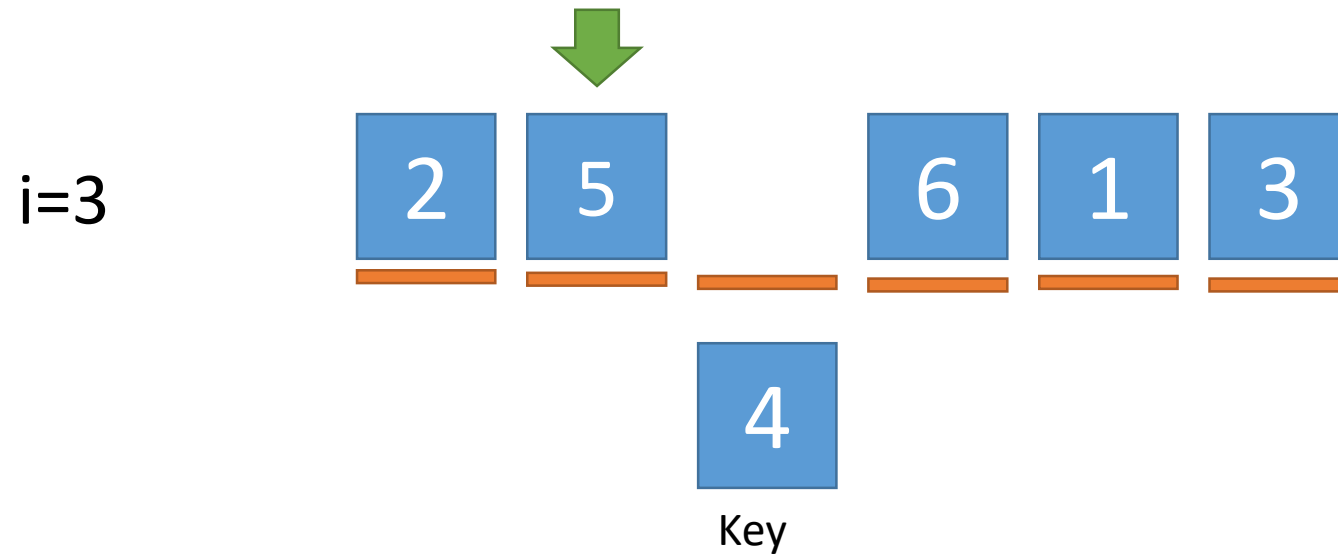
```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1 }  
        A[j+1] = key  
    }  
}
```

Analyzing Insertion Sort



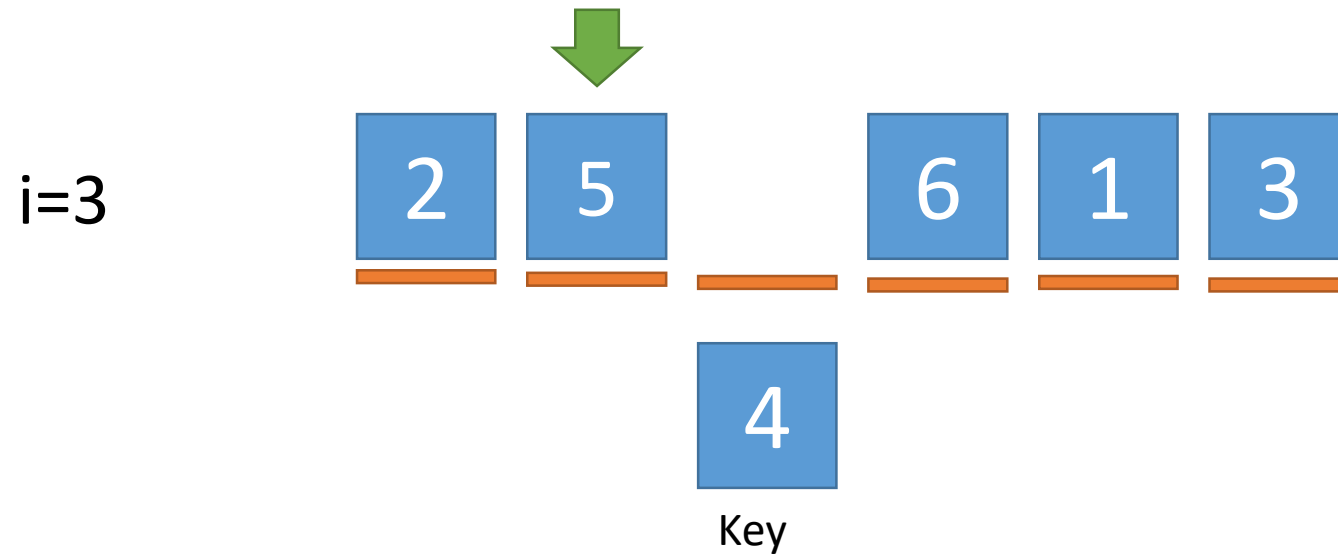
```
InsertionSort(A, n) {  
  for i = 2 to n {  
    → key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1  
    }  
    A[j+1] = key  
  }  
}
```


Analyzing Insertion Sort



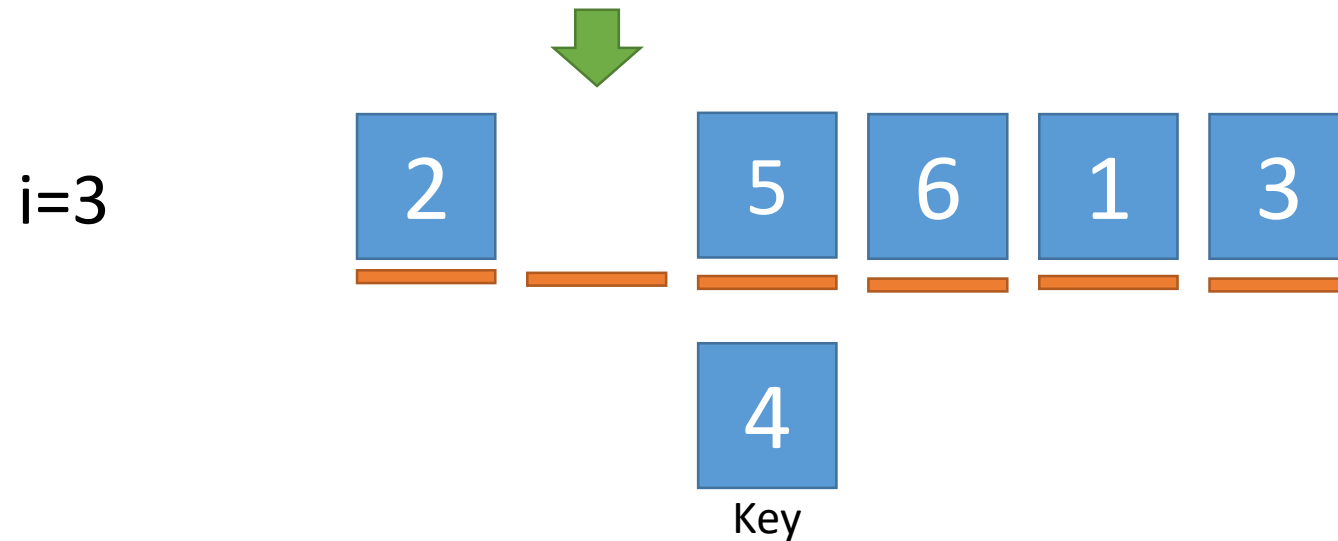
```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        → j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

Analyzing Insertion Sort



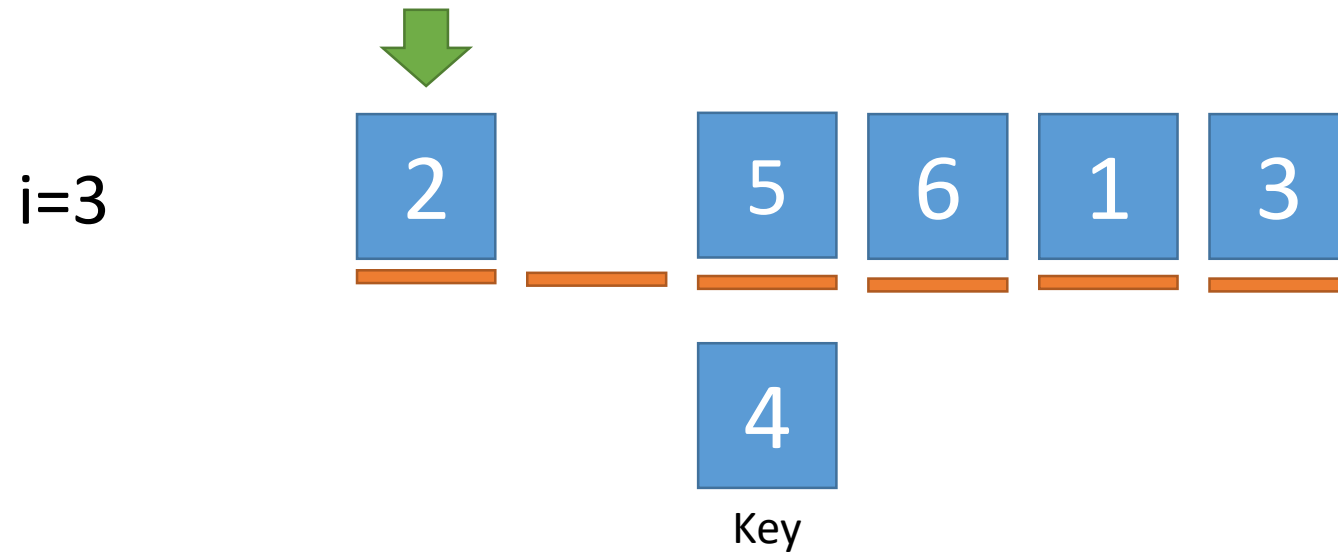
```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    → while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1 }  
    A[j+1] = key  
  }  
}
```

Analyzing Insertion Sort



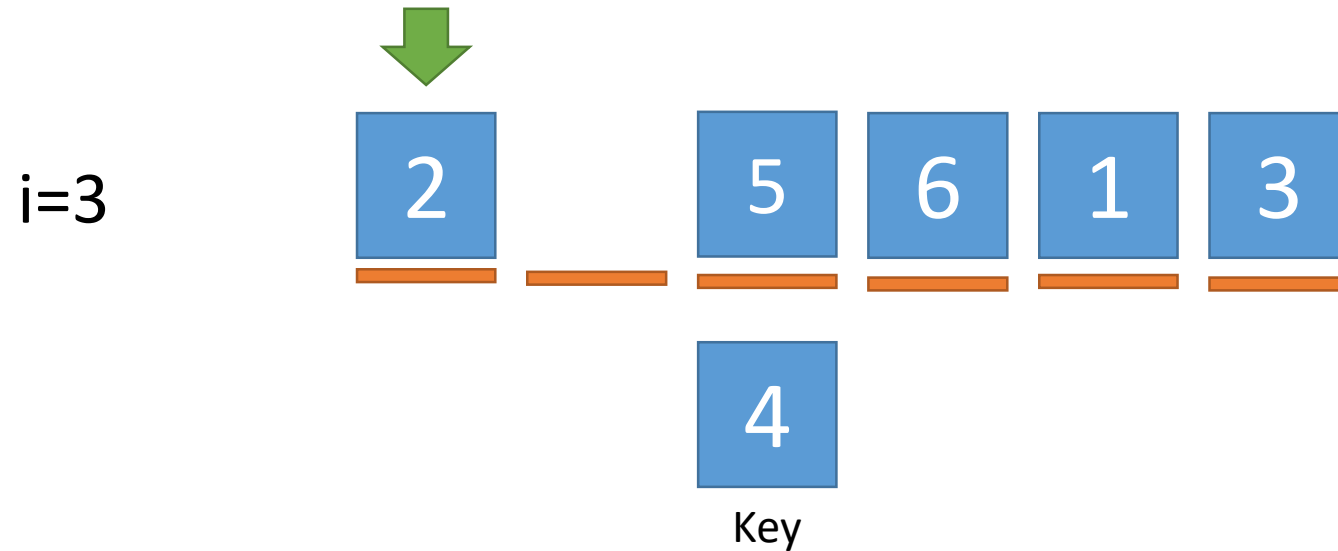
```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    → while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1 }  
    A[j+1] = key  
  }  
}
```

Analyzing Insertion Sort



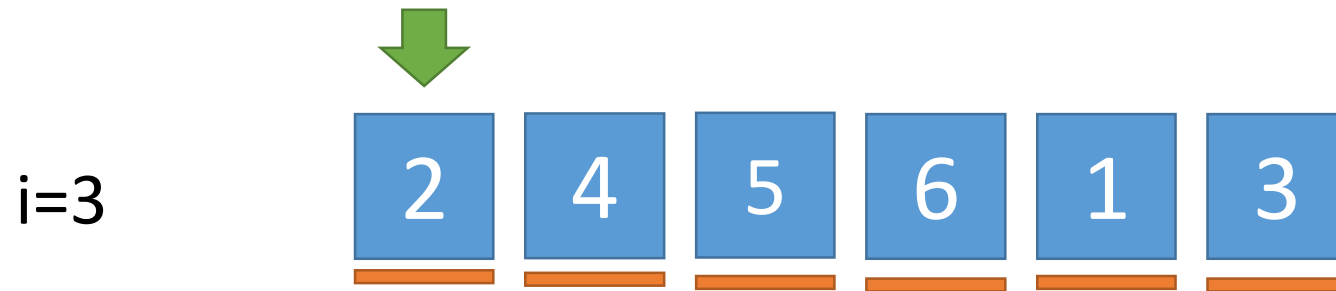
```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1  
    }  
    A[j+1] = key  
  }  
}
```

Analyzing Insertion Sort



```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    → while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1 }  
    A[j+1] = key  
  }  
}
```

Analyzing Insertion Sort



```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1 }  
        A[j+1] = key  
    }  
}
```

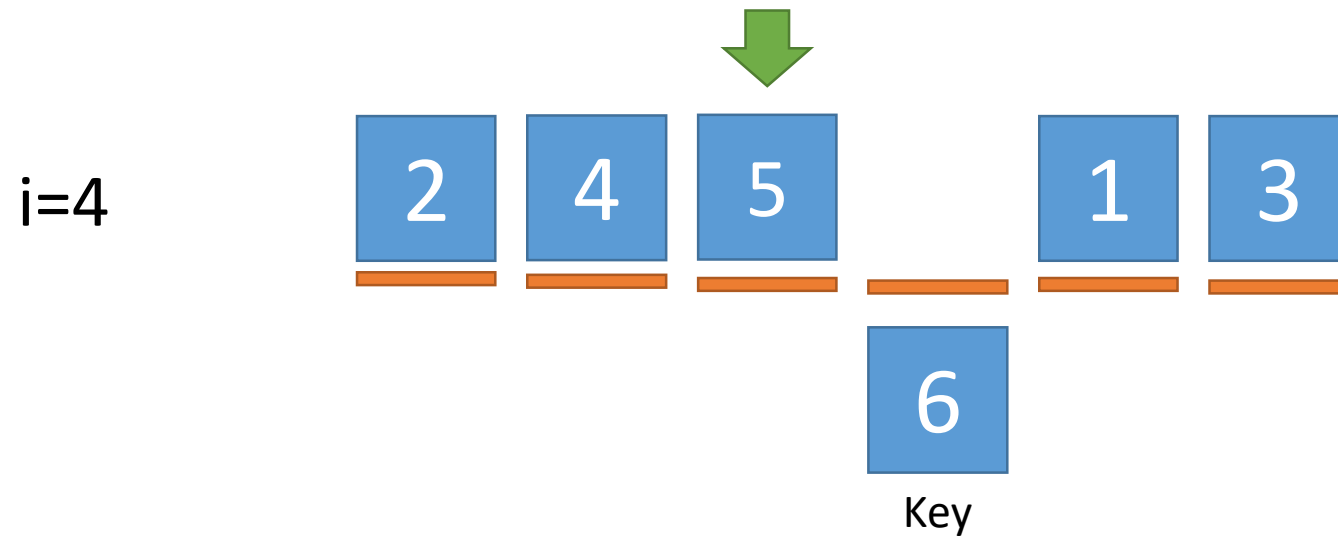
Analyzing Insertion Sort

$i=4$



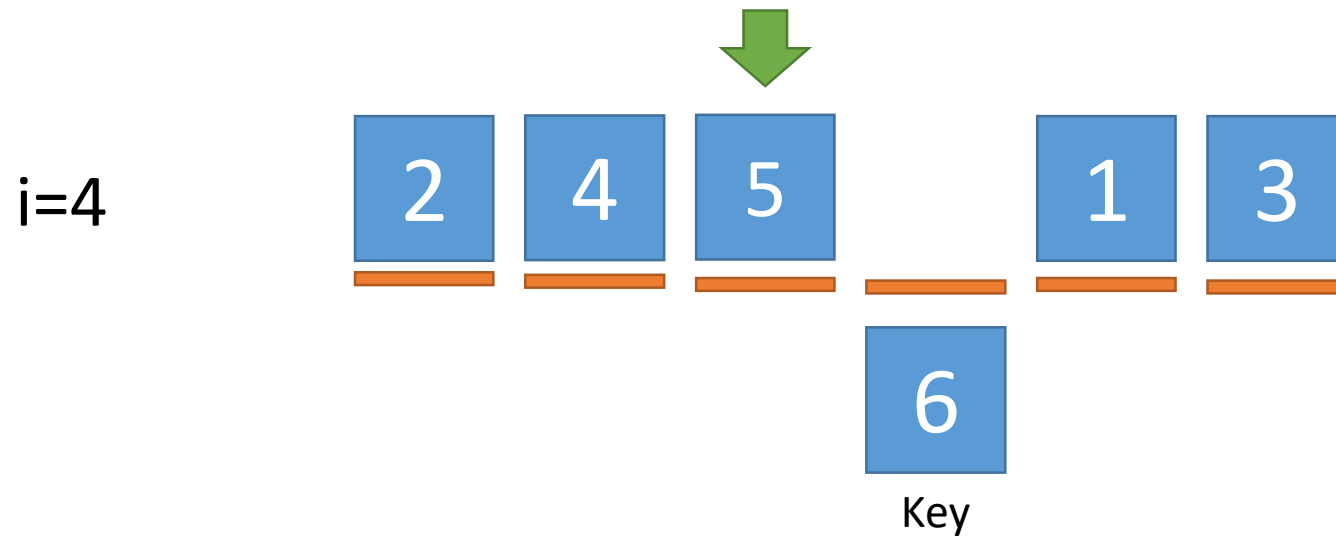
```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

Analyzing Insertion Sort



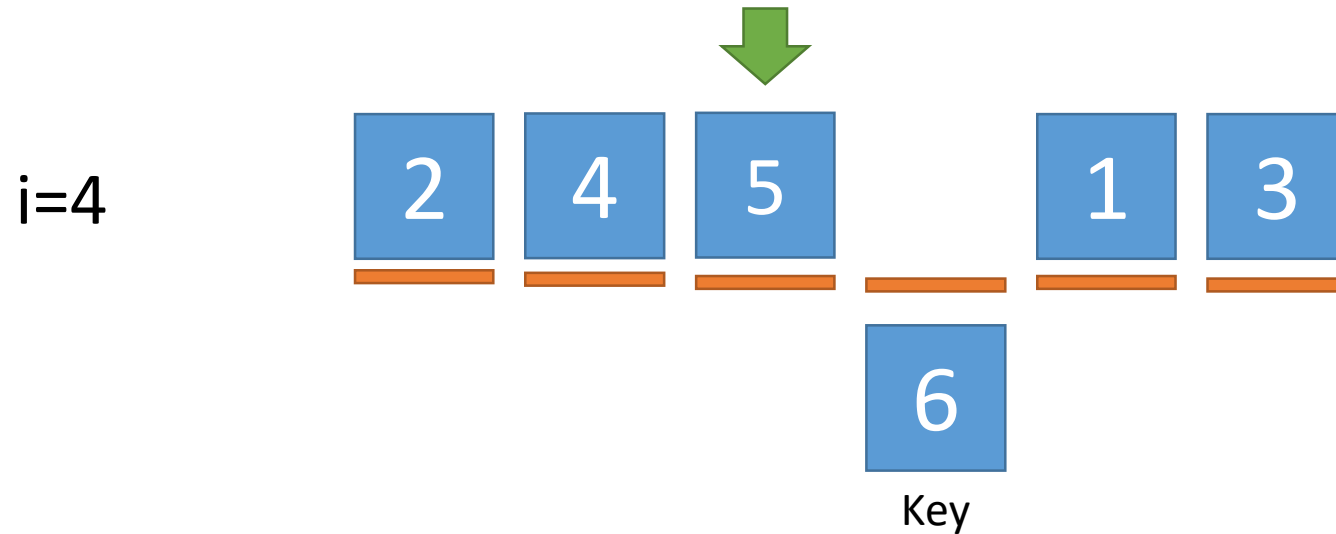
```
InsertionSort(A, n) {  
  for i = 2 to n {  
    → key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1 }  
    A[j+1] = key  
  }  
}
```


Analyzing Insertion Sort



```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        → j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

Analyzing Insertion Sort



```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    → while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1 }  
    A[j+1] = key  
  }  
}
```

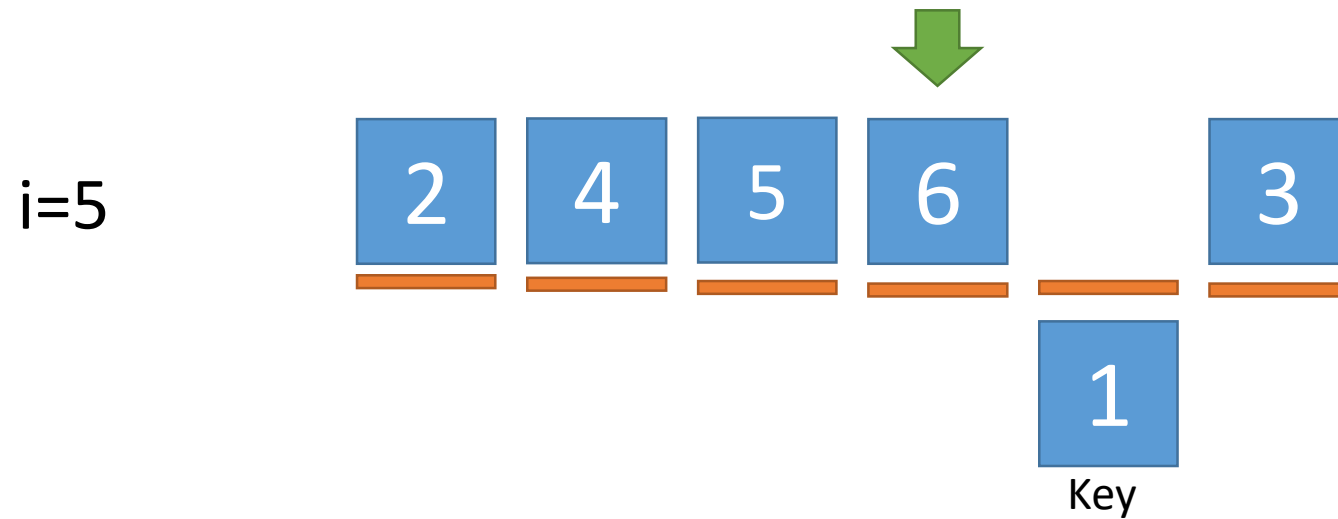
Analyzing Insertion Sort

$i=4$



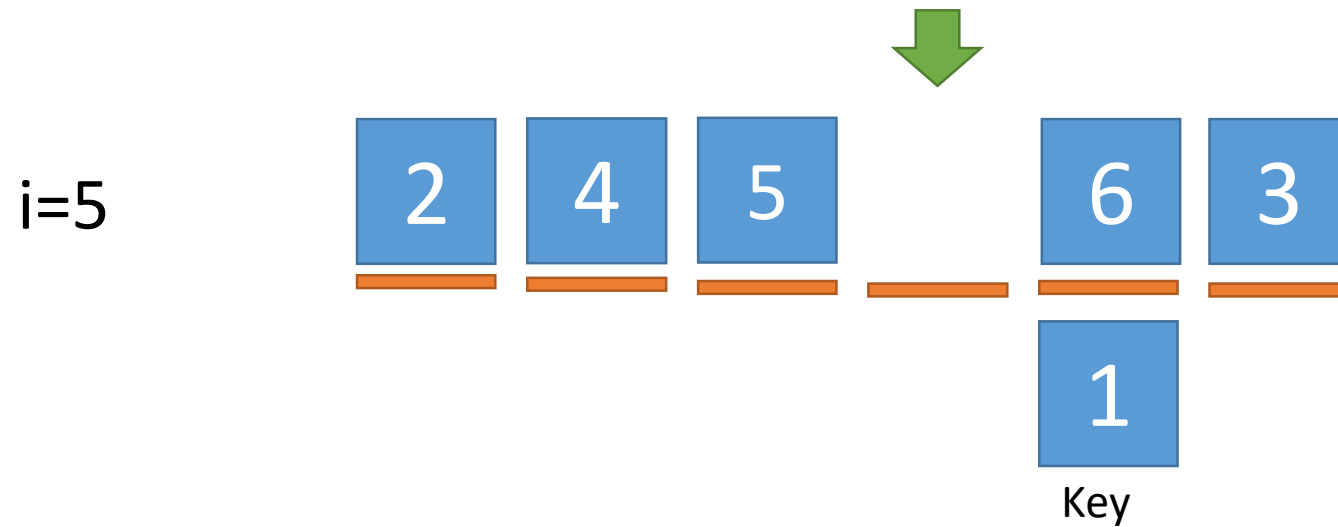
```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1 }  
        → A[j+1] = key  
    }  
}
```

Analyzing Insertion Sort



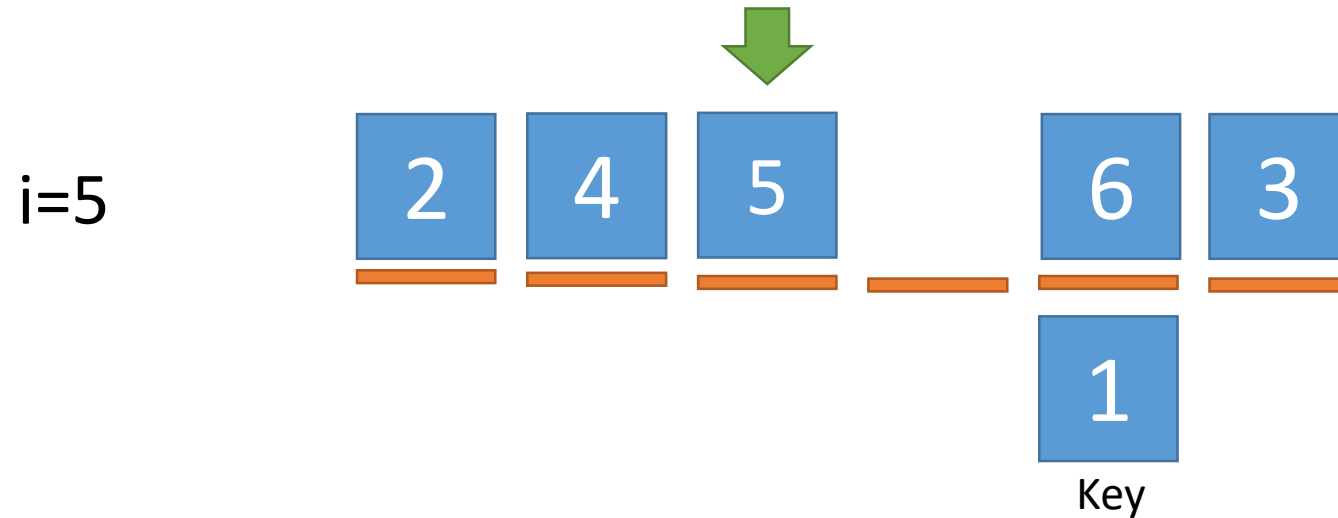
```
InsertionSort(A, n) {  
    → for i = 2 to n {  
        → key = A[i]  
        → j = i - 1;  
        → while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1 }  
        A[j+1] = key  
    }  
}
```

Analyzing Insertion Sort



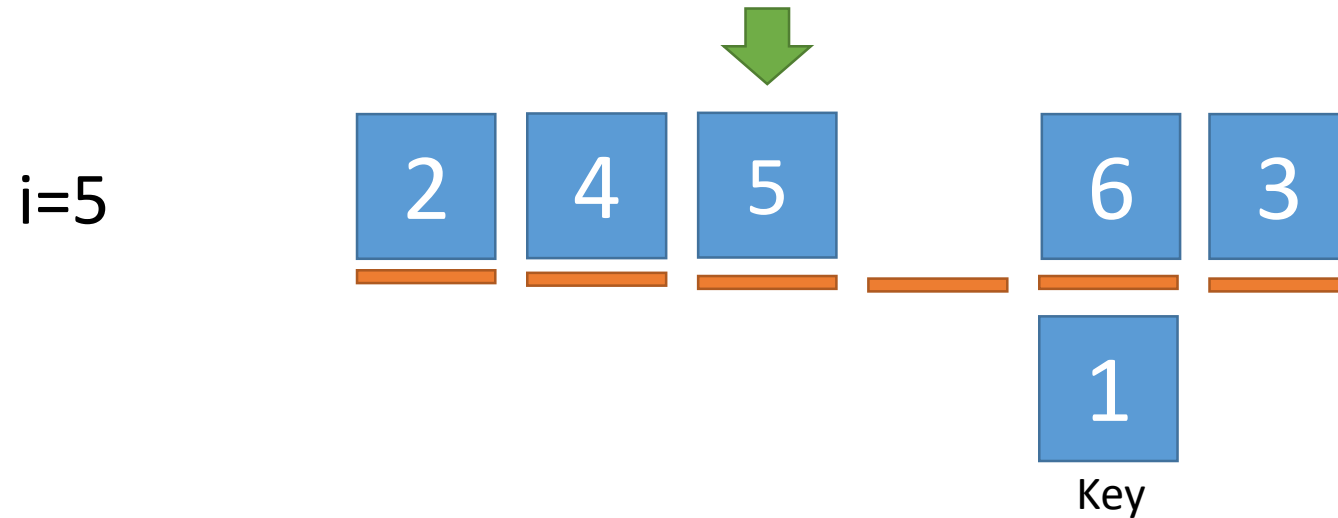
```
InsertionSort(A, n) {  
  for  $i = 2$  to  $n$  {  
     $key = A[i]$   
     $j = i - 1$ ;  
    while ( $j > 0$ ) and ( $A[j] > key$ ) {  
        $A[j+1] = A[j]$   
       $j = j - 1$  }  
     $A[j+1] = key$   
  }  
}
```

Analyzing Insertion Sort



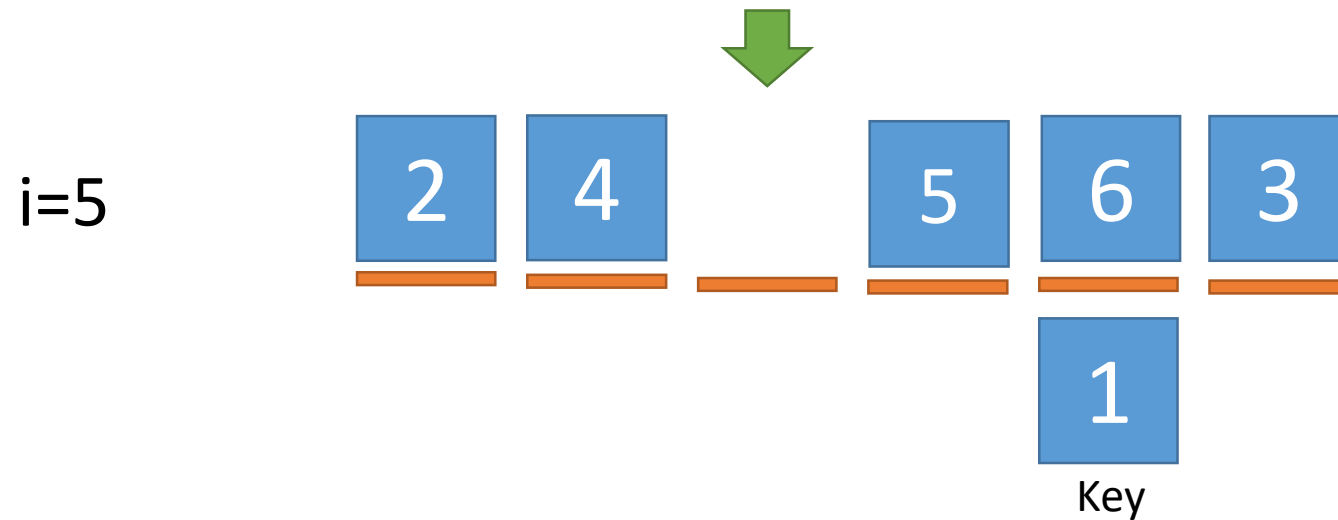
```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1  
    }  
    A[j+1] = key  
  }  
}
```

Analyzing Insertion Sort



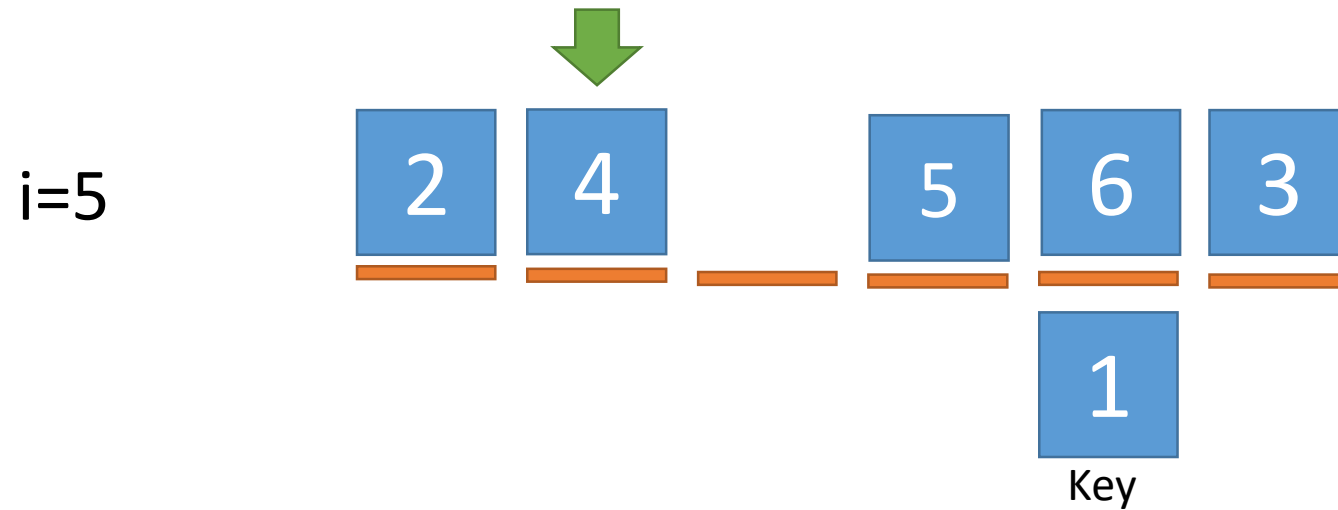
```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    → while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1 }  
    A[j+1] = key  
  }  
}
```

Analyzing Insertion Sort



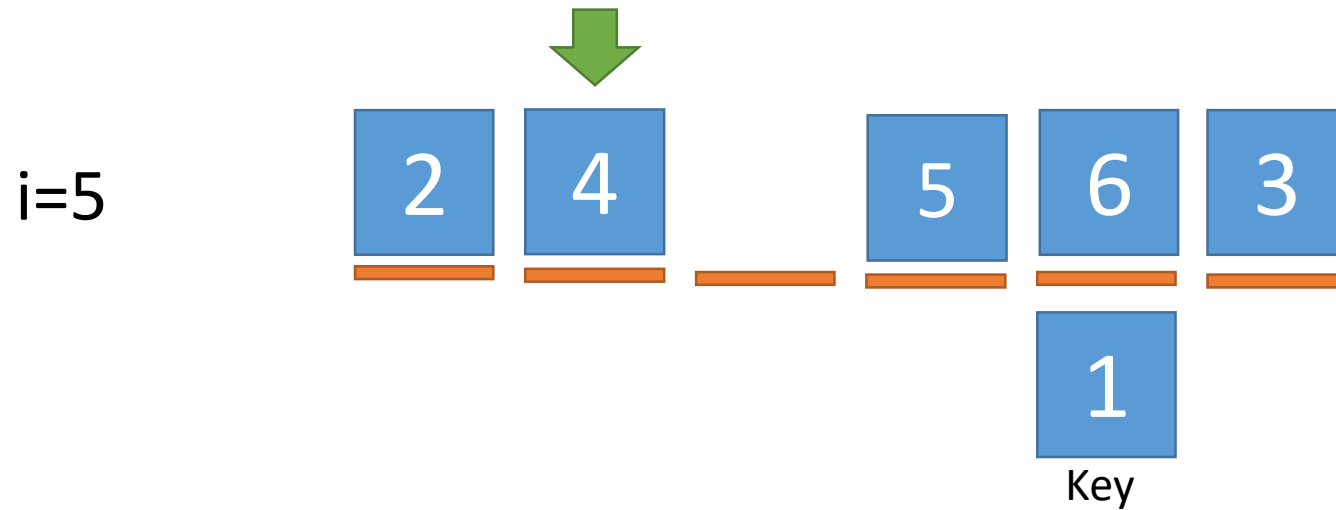
```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
       A[j+1] = A[j]  
      j = j - 1 }  
    A[j+1] = key  
  }  
}
```


Analyzing Insertion Sort



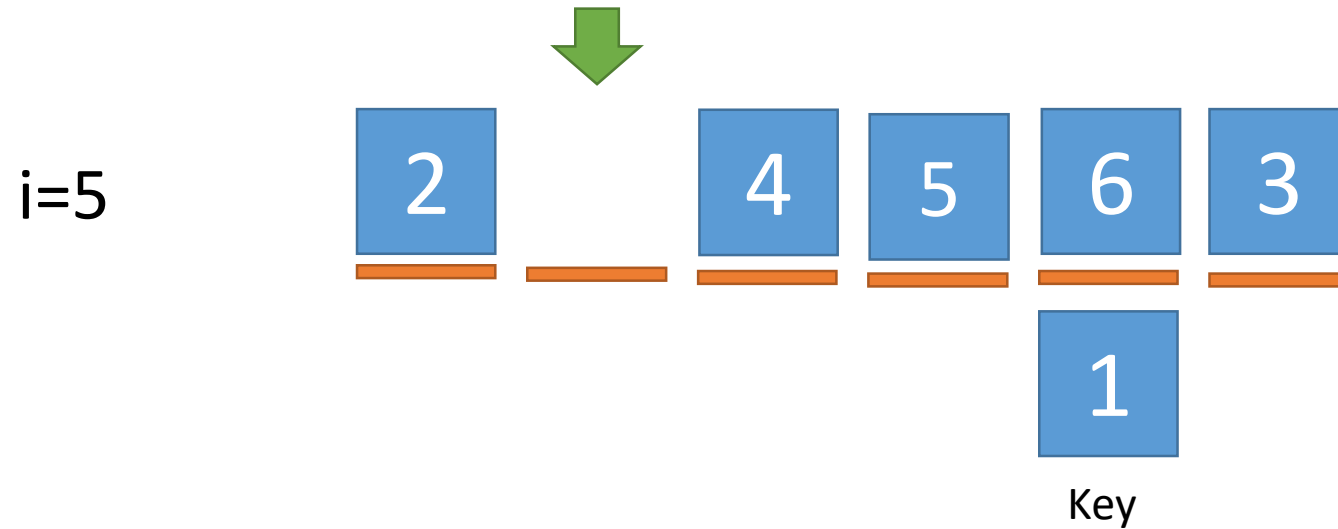
```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      → j = j - 1 }  
    A[j+1] = key  
  }  
}
```

Analyzing Insertion Sort



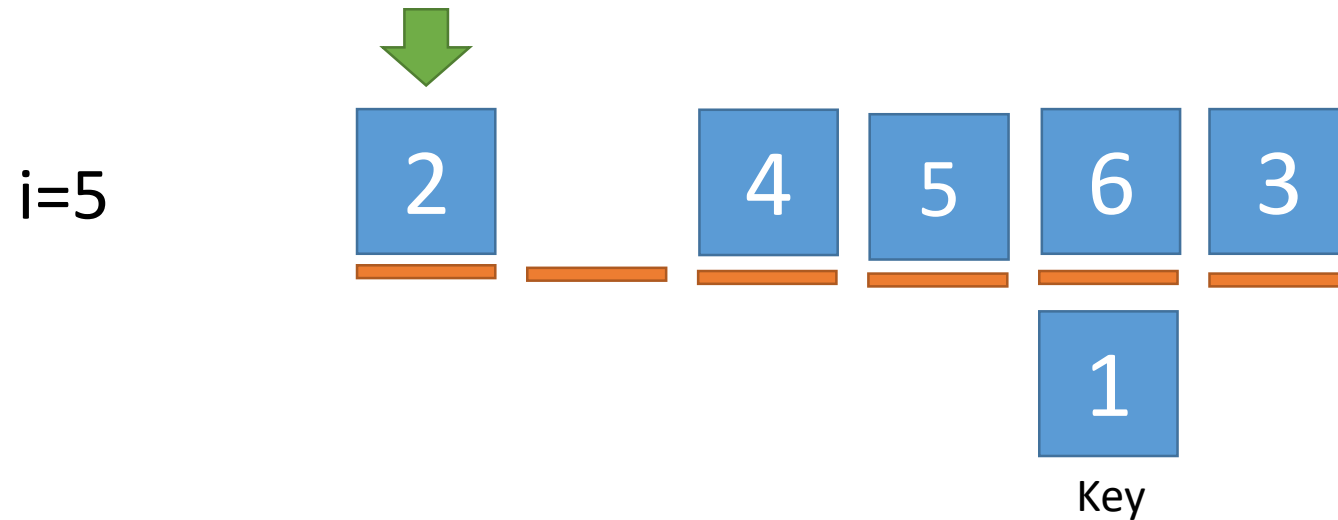
```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        → while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1 }  
        A[j+1] = key  
    }  
}
```

Analyzing Insertion Sort



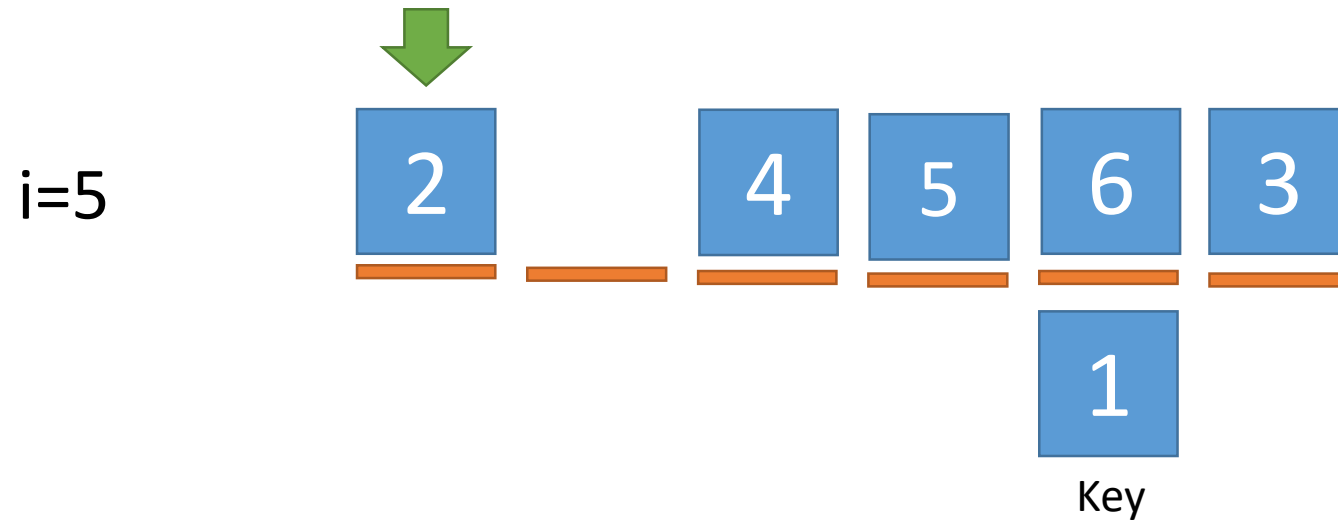
```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
       A[j+1] = A[j]  
      j = j - 1 }  
    A[j+1] = key  
  }  
}
```

Analyzing Insertion Sort



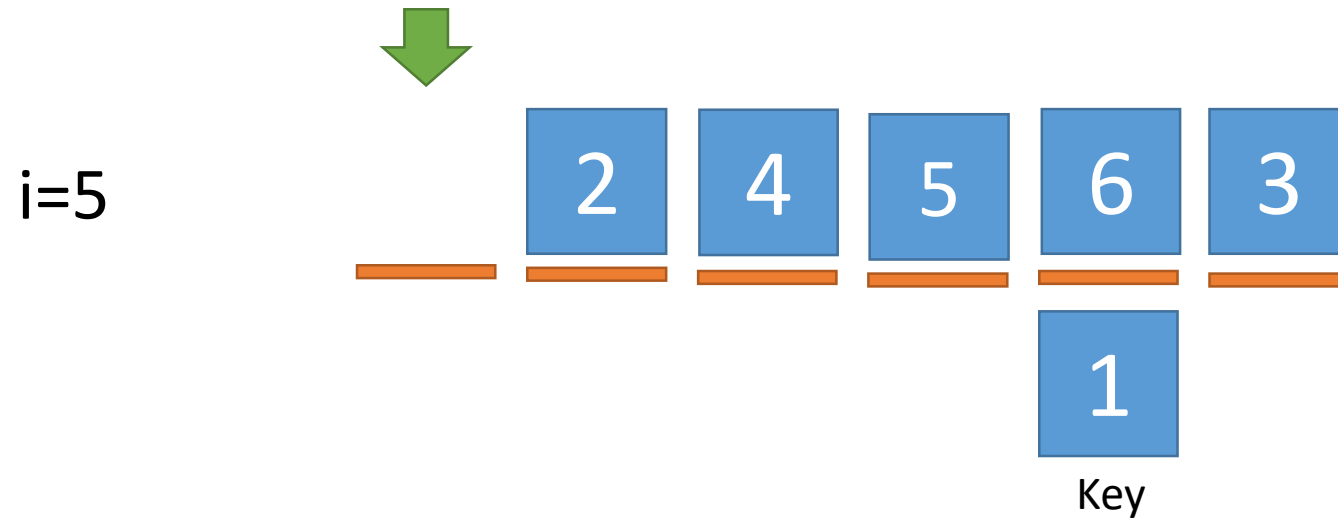
```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
       j = j - 1  
    }  
    A[j+1] = key  
  }  
}
```

Analyzing Insertion Sort



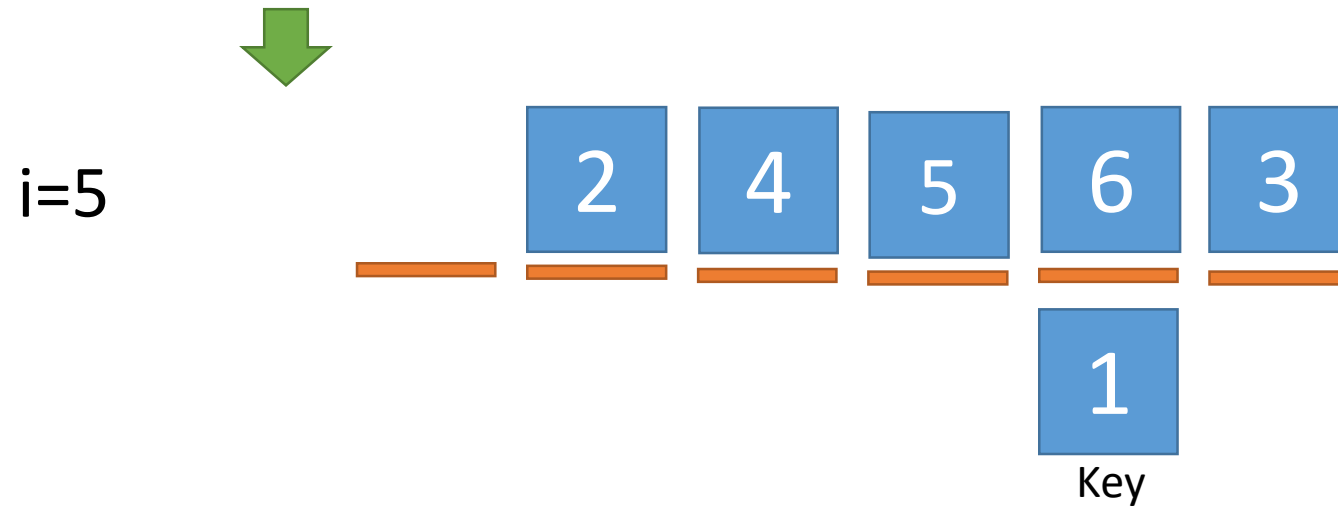
```
InsertionSort(A, n) {  
  for  $i = 2$  to  $n$  {  
     $key = A[i]$   
     $j = i - 1$ ;  
    → while ( $j > 0$ ) and ( $A[j] > key$ ) {  
       $A[j+1] = A[j]$   
       $j = j - 1$  }  
     $A[j+1] = key$   
  }  
}
```

Analyzing Insertion Sort



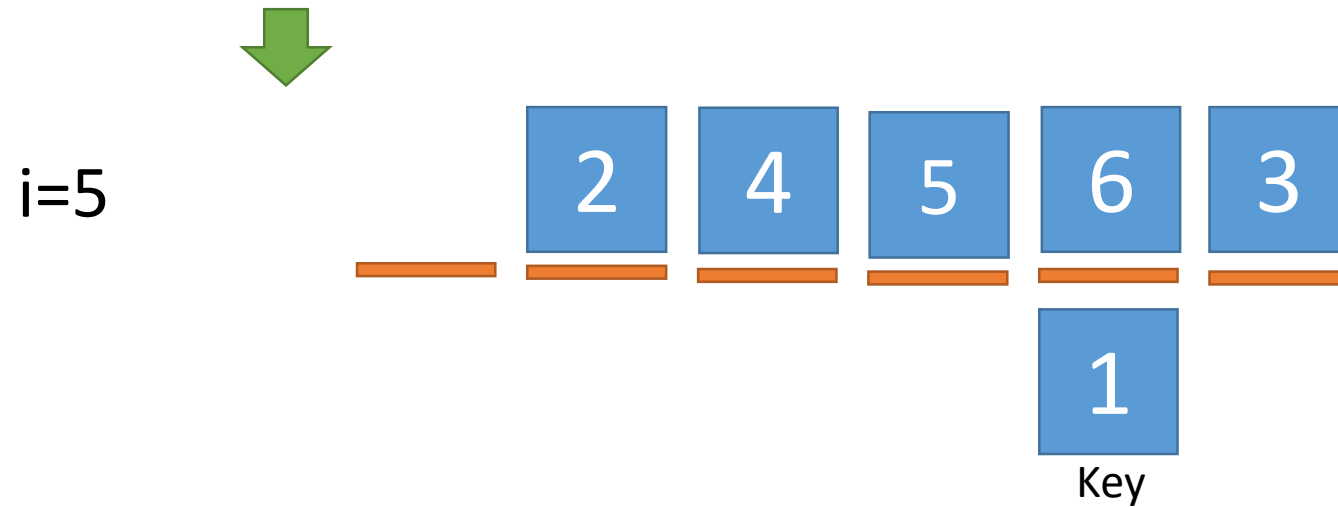
```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      ➔ A[j+1] = A[j]  
      j = j - 1 }  
    A[j+1] = key  
  }  
}
```

Analyzing Insertion Sort



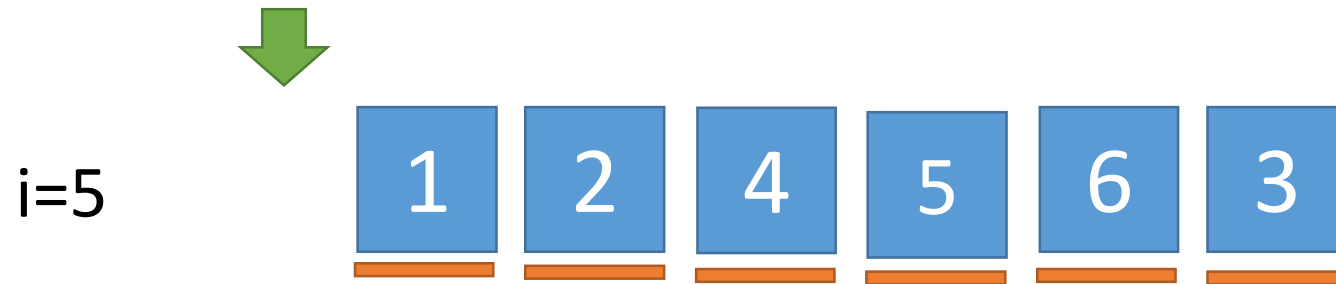
```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
       j = j - 1  
    }  
    A[j+1] = key  
  }  
}
```

Analyzing Insertion Sort



```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    → while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1 }  
    A[j+1] = key  
  }  
}
```


Analyzing Insertion Sort



```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1 }  
    → A[j+1] = key  
  }  
}
```

Analyzing Insertion Sort

DO IT YOURSELF!!

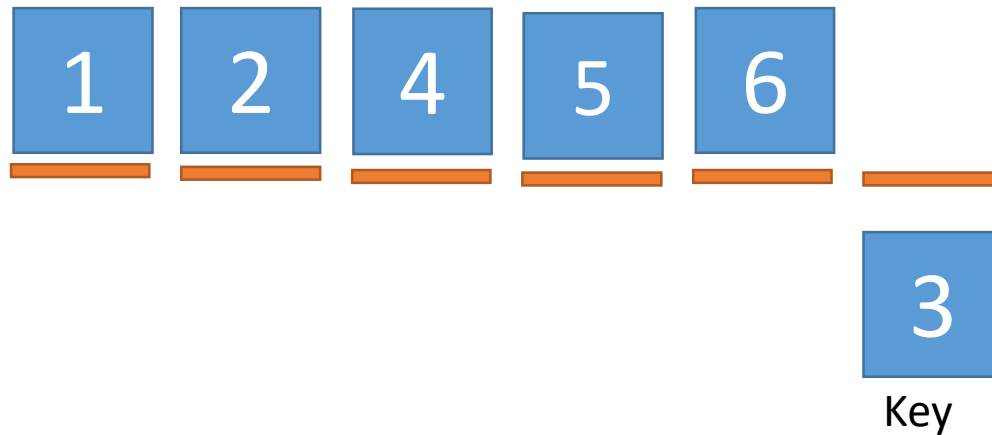
$i=6$



```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1 }  
    A[j+1] = key  
  }  
}
```

Analyzing Insertion Sort

$i=6$

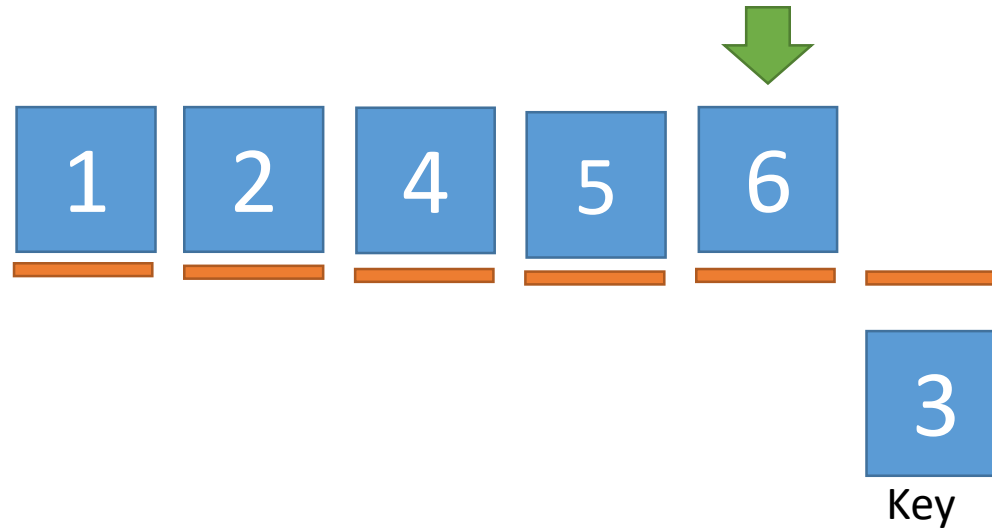


DO IT YOURSELF!!

```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1 }  
    A[j+1] = key  
  }  
}
```

Analyzing Insertion Sort

$i=6$

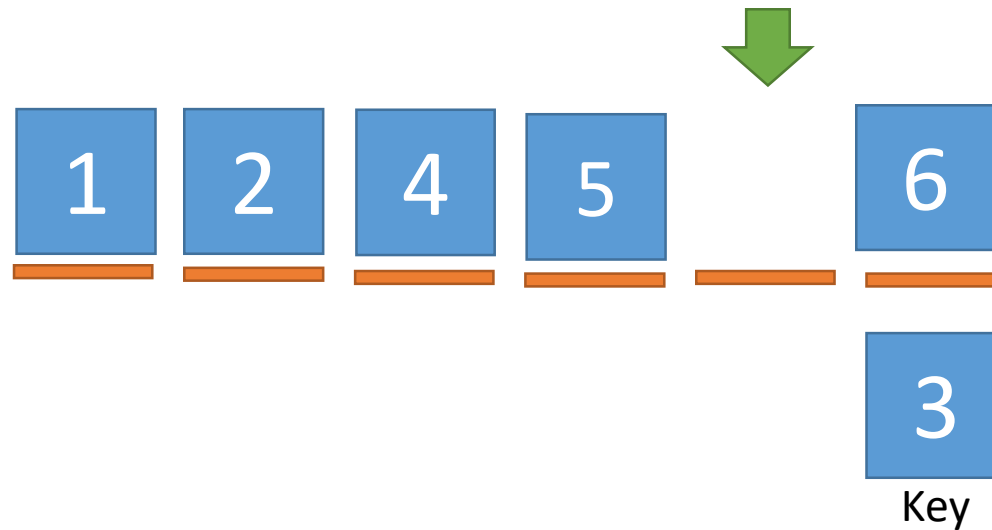


DO IT YOURSELF!!

```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1 }  
    A[j+1] = key  
  }  
}
```

Analyzing Insertion Sort

$i=6$

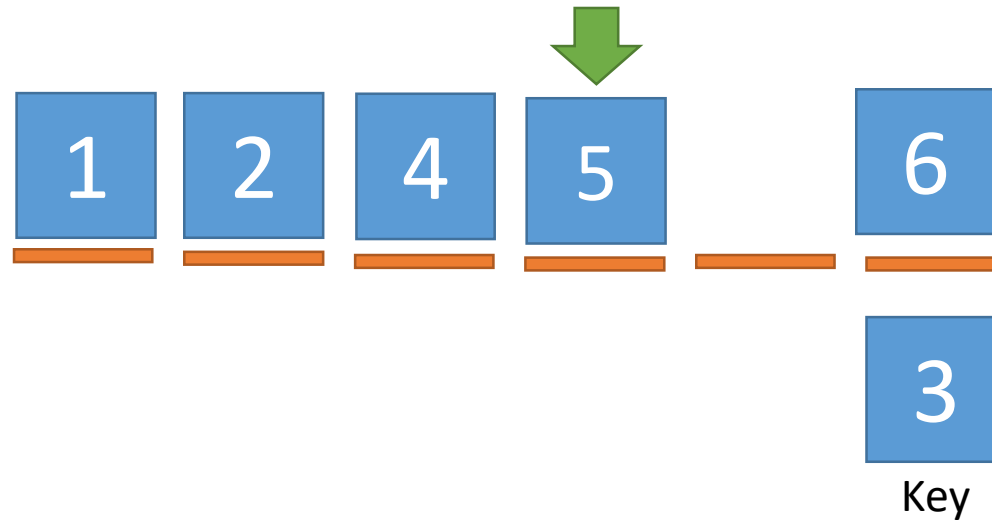


DO IT YOURSELF!!

```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1 }  
    A[j+1] = key  
  }  
}
```

Analyzing Insertion Sort

$i=6$

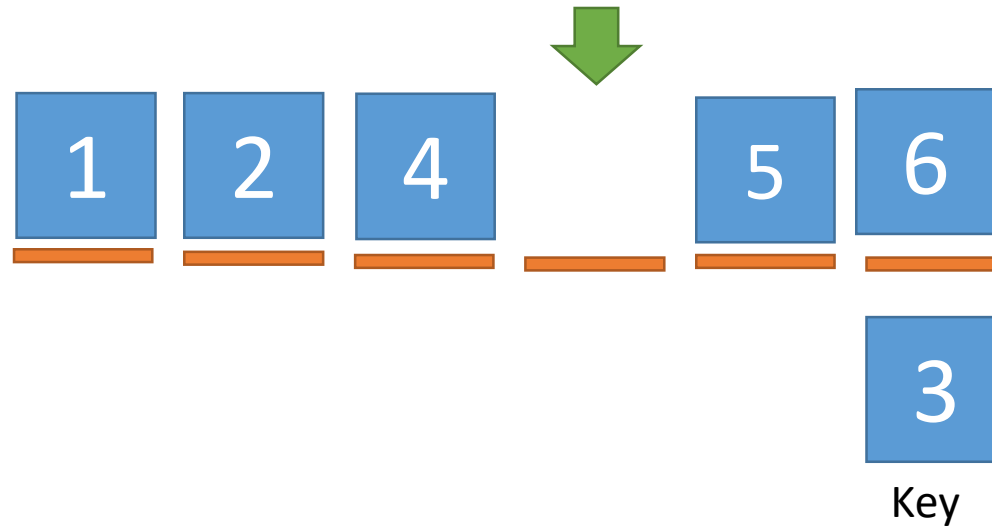


DO IT YOURSELF!!

```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1 }  
    A[j+1] = key  
  }  
}
```

Analyzing Insertion Sort

$i=6$

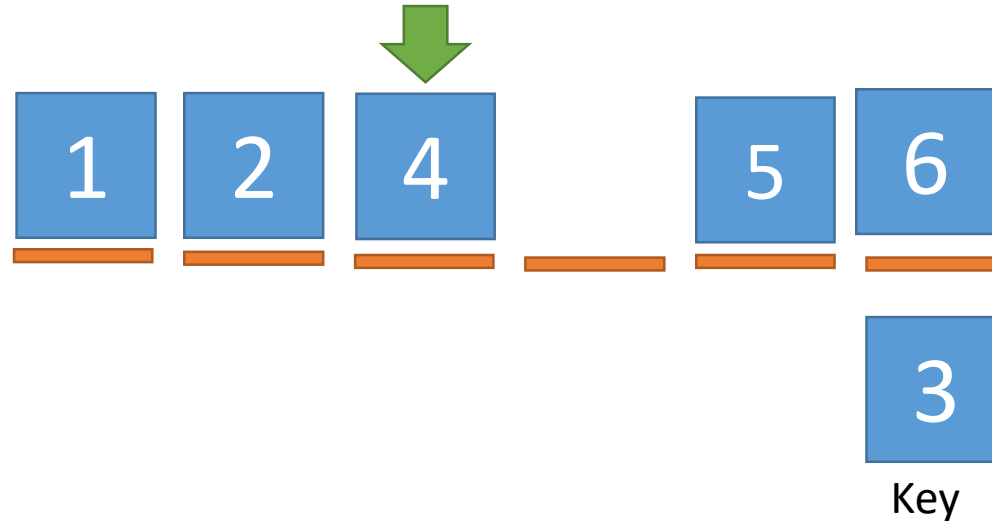


DO IT YOURSELF!!

```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1 }  
    A[j+1] = key  
  }  
}
```

Analyzing Insertion Sort

i=6

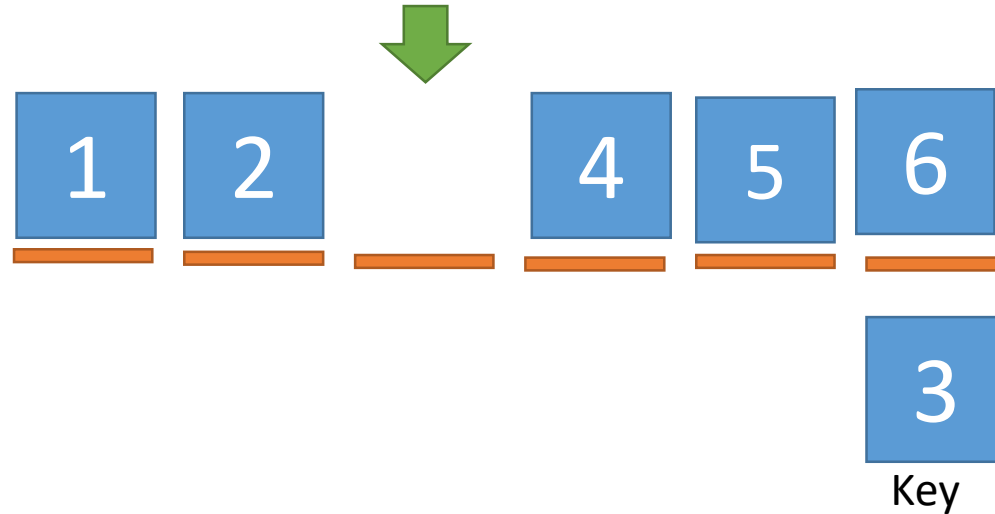


DO IT YOURSELF!!

```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1 }  
    A[j+1] = key  
  }  
}
```


Analyzing Insertion Sort

$i=6$

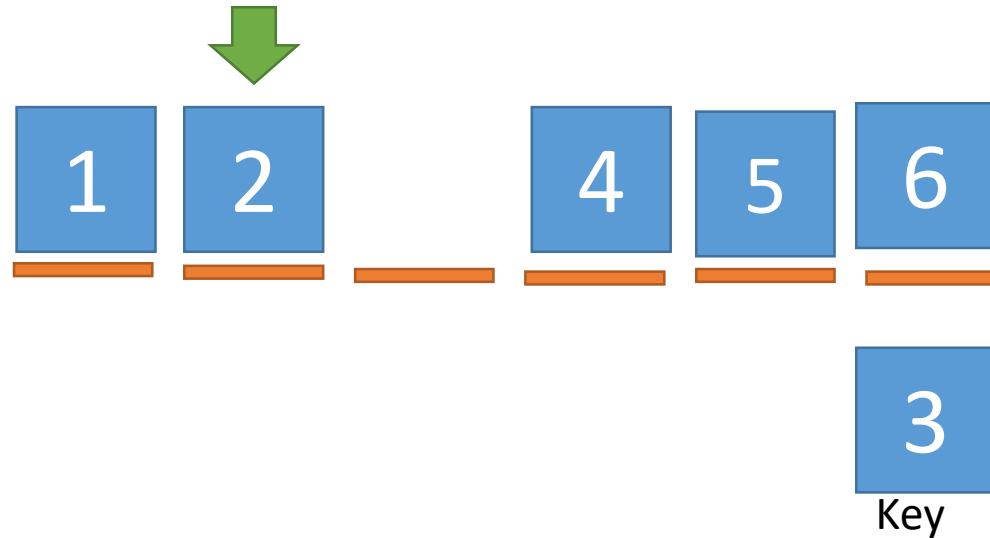


DO IT YOURSELF!!

```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1 }  
    A[j+1] = key  
  }  
}
```

Analyzing Insertion Sort

$i=6$



DO IT YOURSELF!!

```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1 }  
    A[j+1] = key  
  }  
}
```

Analyzing Insertion Sort

$i=6$



DO IT YOURSELF!!

```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1 }  
    A[j+1] = key  
  }  
}
```

Analyzing Insertion Sort

DO IT YOURSELF!!

$i=6$



NOTE: This loop will run n (6) times

```
InsertionSort(A, n) {  
  for  $i = 2$  to  $n$  {  
     $key = A[i]$   
     $j = i - 1$ ;  
    while ( $j > 0$ ) and ( $A[j] > key$ ) {  
       $A[j+1] = A[j]$   
       $j = j - 1$  }  
     $A[j+1] = key$   
  }  
}
```

Analyzing Insertion Sort

DO IT YOURSELF!!

$i=6$



NOTE: This loop will run n (6) times

```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1  
    }  
    A[j+1] = key  
  }  
}
```

Analyzing Insertion Sort

DO IT YOURSELF!!

$i=6$



NOTE: This loop will run n (6) times

```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1 }  
    A[j+1] = key  
  }  
}
```



Analyzing Insertion Sort

Keep in mind

- Each line (statement) in a code takes some time to run.
- It will be different for different PCs
- Let us call this time required to execute a statement : **cost**
- Let us analyze the **time required** for insertion sort

Analyzing Insertion Sort

Let us try to determine the total time

Analyzing Insertion Sort

Statement	Cost	Times
InsertionSort(A, n) {		
for i = 2 to n {	c_1	n
key = A[i]	c_2	$(n - 1)$
j = i - 1;	c_3	$(n - 1)$
while (j > 0) and (A[j] > key) {	c_4	T
A[j+1] = A[j]	c_5	$(T - (n - 1))$
j = j - 1 }	c_6	$(T - (n - 1))$
A[j+1] = key	c_7	$(n - 1)$
}		
}		

$T = t_2 + t_3 + \dots + t_n$, where t_i is the number of while expression evaluations for the i^{th} for loop iteration

Analyzing Insertion Sort

$$\begin{aligned} T(n) &= c_1n + c_2(n-1) + c_3(n-1) + c_4T + c_5(T - (n-1)) + c_6(T - (n-1)) \\ &\quad + c_7(n-1) \\ &= c_8T + c_9n + c_{10} \end{aligned}$$

Analyzing Insertion Sort

$$\begin{aligned} T(n) &= c_1n + c_2(n-1) + c_3(n-1) + c_4T + c_5(T - (n-1)) + c_6(T - (n-1)) \\ &\quad + c_7(n-1) \\ &= c_8T + c_9n + c_{10} \end{aligned}$$

- We can understand that
 - The time will be more if the number of input elements is more
 - We can also understand that the time will be more if the while loop runs more

Analyzing Insertion Sort

$$T(n) = c_8T + c_9n + c_{10}$$

We can also understand that the time will be more if the while loop runs more

What can T be?

- **Best case:** the array is sorted (inner loop body never executed)
 - ◆ $t_i = 1 \Rightarrow T = n-1$
 - ◆ $T(n) = an + b$, a linear function of n
- **Worst case:** the array is reverse sorted (inner loop body executed for all previous elements)
 - ◆ $t_i = i \Rightarrow T = n(n+1)/2 - 1$
 - ◆ $T(n) = an^2 + bn + c$, a quadratic function of n
- **Average case:**
 - ◆ ???

Best Case

Statement	Cost	Times
InsertionSort(A, n) {		
for i = 2 to n {	c_1	n
key = A[i]	c_2	$(n - 1)$
j = i - 1;	c_3	$(n - 1)$
while (j > 0) and (A[j] > key) {	c_4	T
A[j+1] = A[j]	c_5	$(T - (n - 1))$
j = j - 1 }	c_6	$(T - (n - 1))$
A[j+1] = key	c_7	$(n - 1)$
}		
}		

$T = t_2 + t_3 + \dots + t_n$, where t_i is the number of while expression evaluations for the i^{th} for loop iteration

WHEN THE ARRAY IS FULLY SORTED
{1,2,3,5,7,9}

$$T(n) = an + b$$

Worst Case

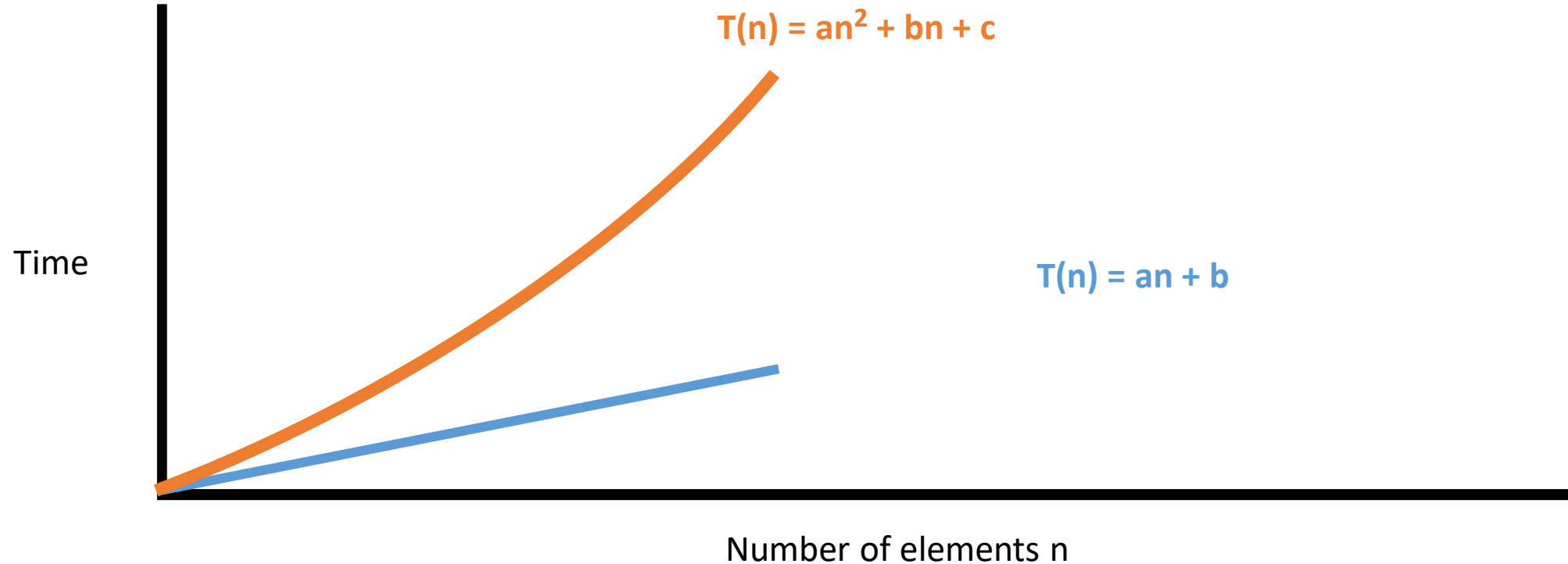
Statement	Cost	Times
InsertionSort(A, n) {		
for i = 2 to n {	c_1	n
key = A[i]	c_2	$(n - 1)$
j = i - 1;	c_3	$(n - 1)$
while (j > 0) and (A[j] > key) {	c_4	T
A[j+1] = A[j]	c_5	$(T - (n - 1))$
j = j - 1 }	c_6	$(T - (n - 1))$
A[j+1] = key	c_7	$(n - 1)$
}		
}		

$T = t_2 + t_3 + \dots + t_n$, where t_i is the number of while expression evaluations for the i^{th} for loop iteration

WHEN THE ARRAY IS IN REVERSE ORDER
 $\{5, 4, 3, 1\}$

$$T(n) = an^2 + bn + c$$

What does the best case and the worst case imply?





ASYMPTOTIC ANALYSIS

Asymptotic Analysis

Worst case

- Provides an upper bound on running time
- An absolute guarantee

Average case

- Provides the expected running time
- Very useful, but treat with care: what is “average”?
 - ◆ Random (equally likely) inputs
 - ◆ Real-life inputs

Best case

Upper Bound Notation

We say InsertionSort's run time is $O(n^2)$

- Properly we should say run time is *in* $O(n^2)$
- Read O as “Big- O ” (you’ll also hear it as “order”)

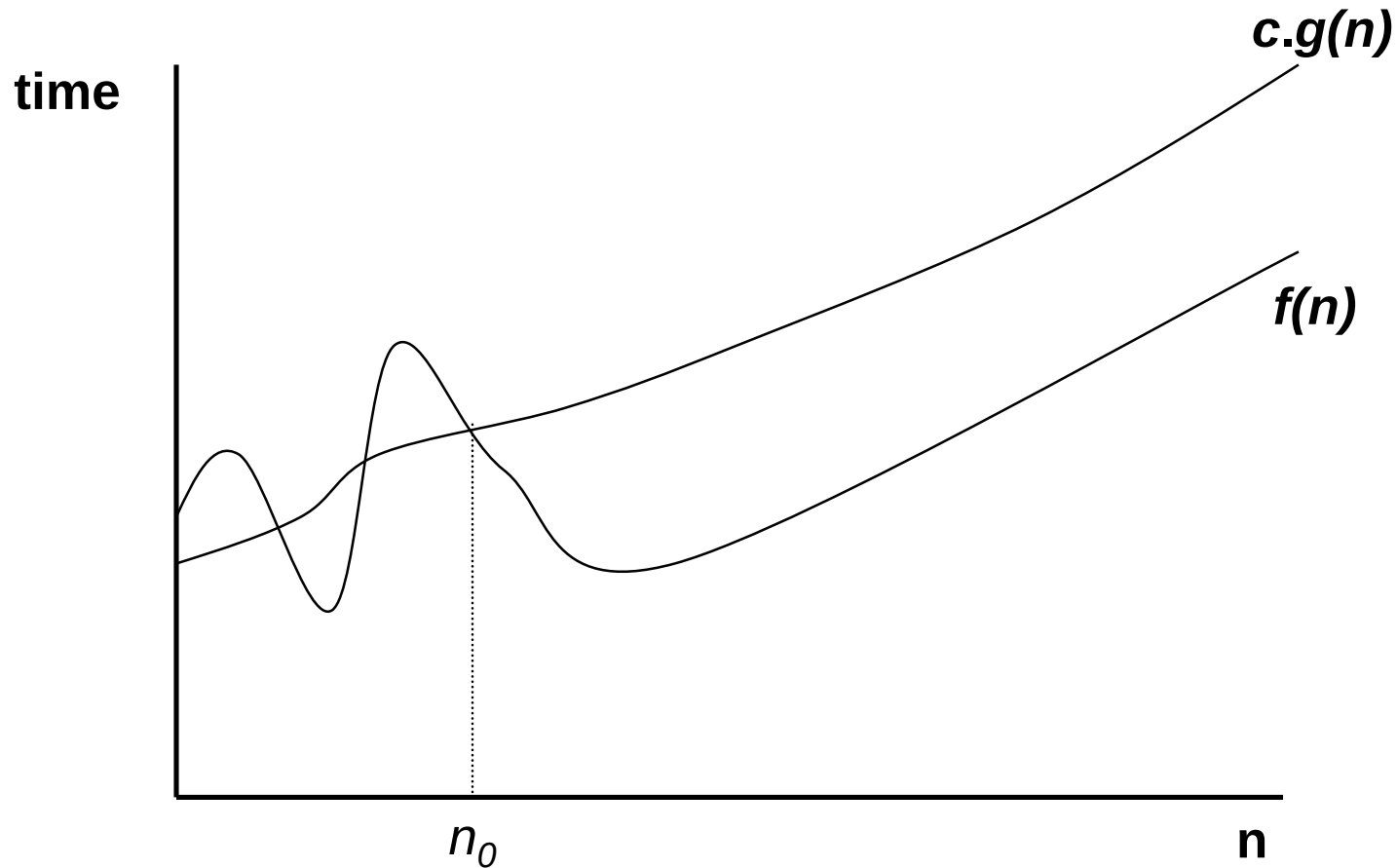
In general a function

- $f(n)$ is $O(g(n))$ if there exist positive constants c and n_0 such that $0 \leq f(n) \leq c \cdot g(n)$ for all $n \geq n_0$

Formally

- $O(g(n)) = \{ f(n) : \exists \text{ positive constants } c \text{ and } n_0 \text{ such that } 0 \leq f(n) \leq c \cdot g(n) \forall n \geq n_0 \}$

Upper Bound Notation



We say $g(n)$ is an *asymptotic upper bound* for $f(n)$

Lower Bound Notation

We say InsertionSort's run time is $\Omega(n)$

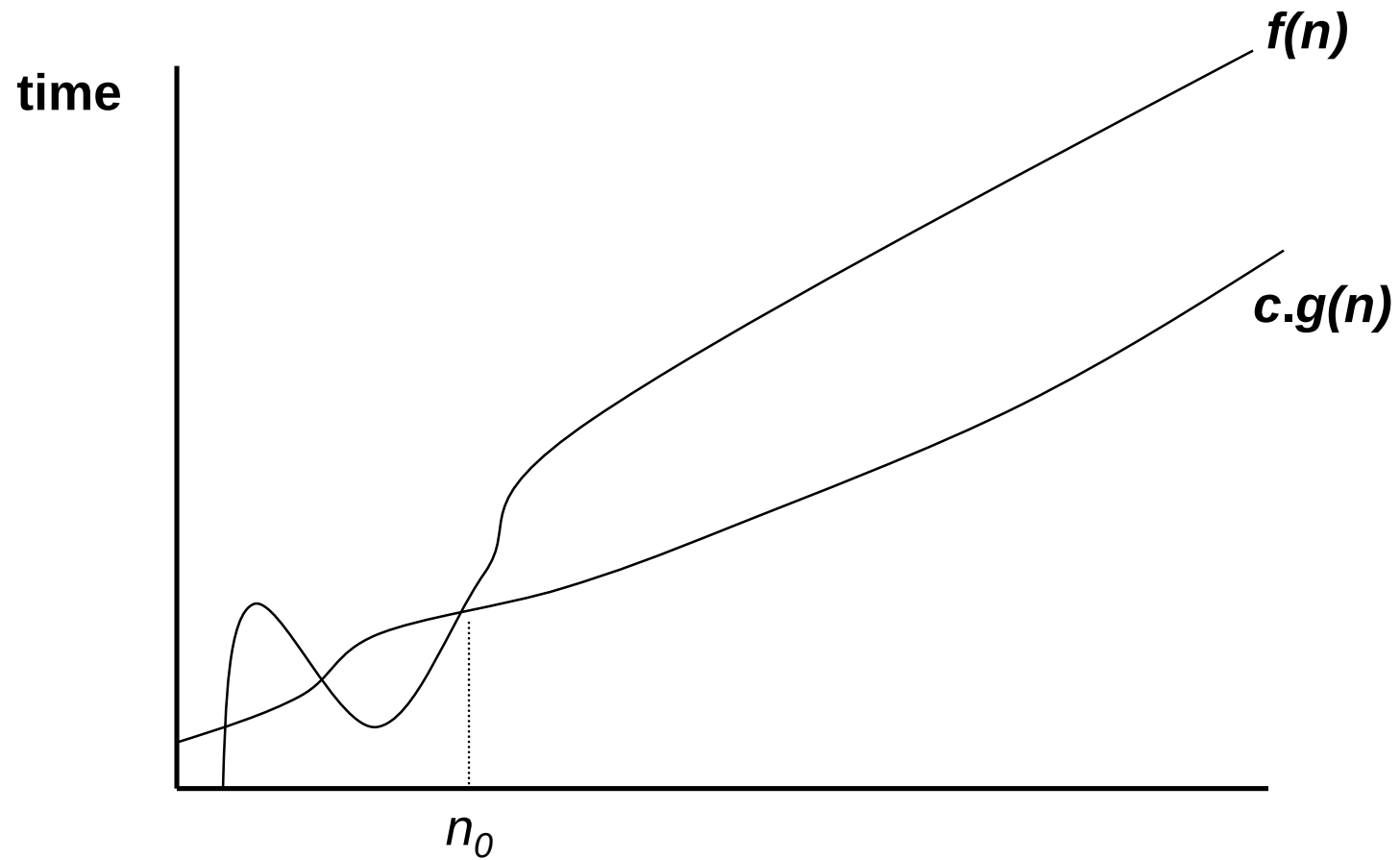
In general a function

- $f(n)$ is $\Omega(g(n))$ if $\exists$ positive constants c and n_0 such that $0 \leq c \cdot g(n) \leq f(n) \quad \forall n \geq n_0$

Proof:

- Suppose run time is $an + b$
 - ◆ Assume a and b are positive
- $an \leq an + b$

Lower Bound Notation



Asymptotic Tight Bound Notation

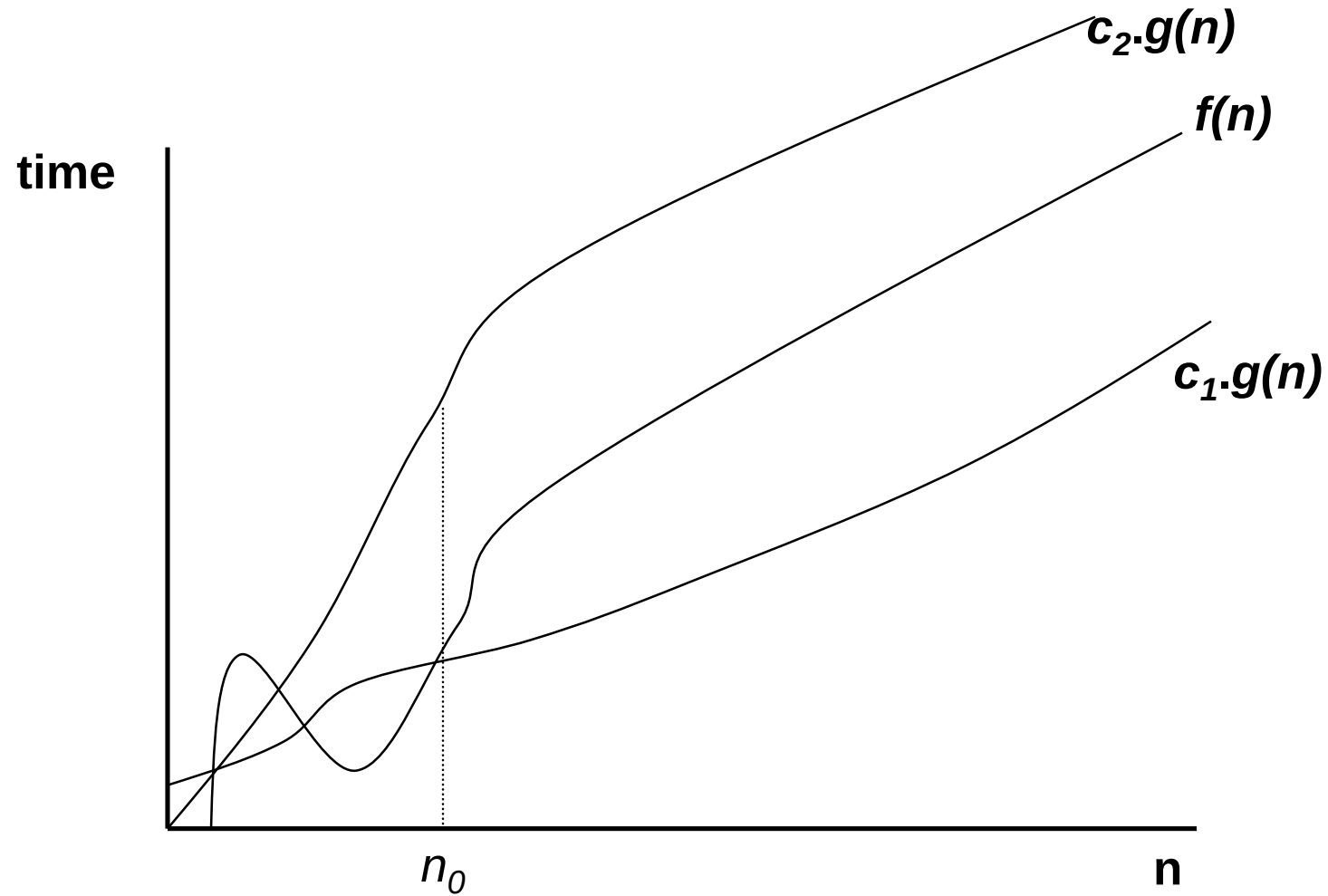
A function $f(n)$ is $\Theta(g(n))$ if $\exists$ positive constants c_1 , c_2 , and n_0 such that

$$0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \quad \forall n \geq n_0$$

Theorem

- $f(n)$ is $\Theta(g(n))$ iff $f(n)$ is both $O(g(n))$ and $\Omega(g(n))$
- Proof:

Asymptotic Tight Bound Notation





NEXT CLASS

A bit more about asymptotic notations and more on other algorithms



Thank You